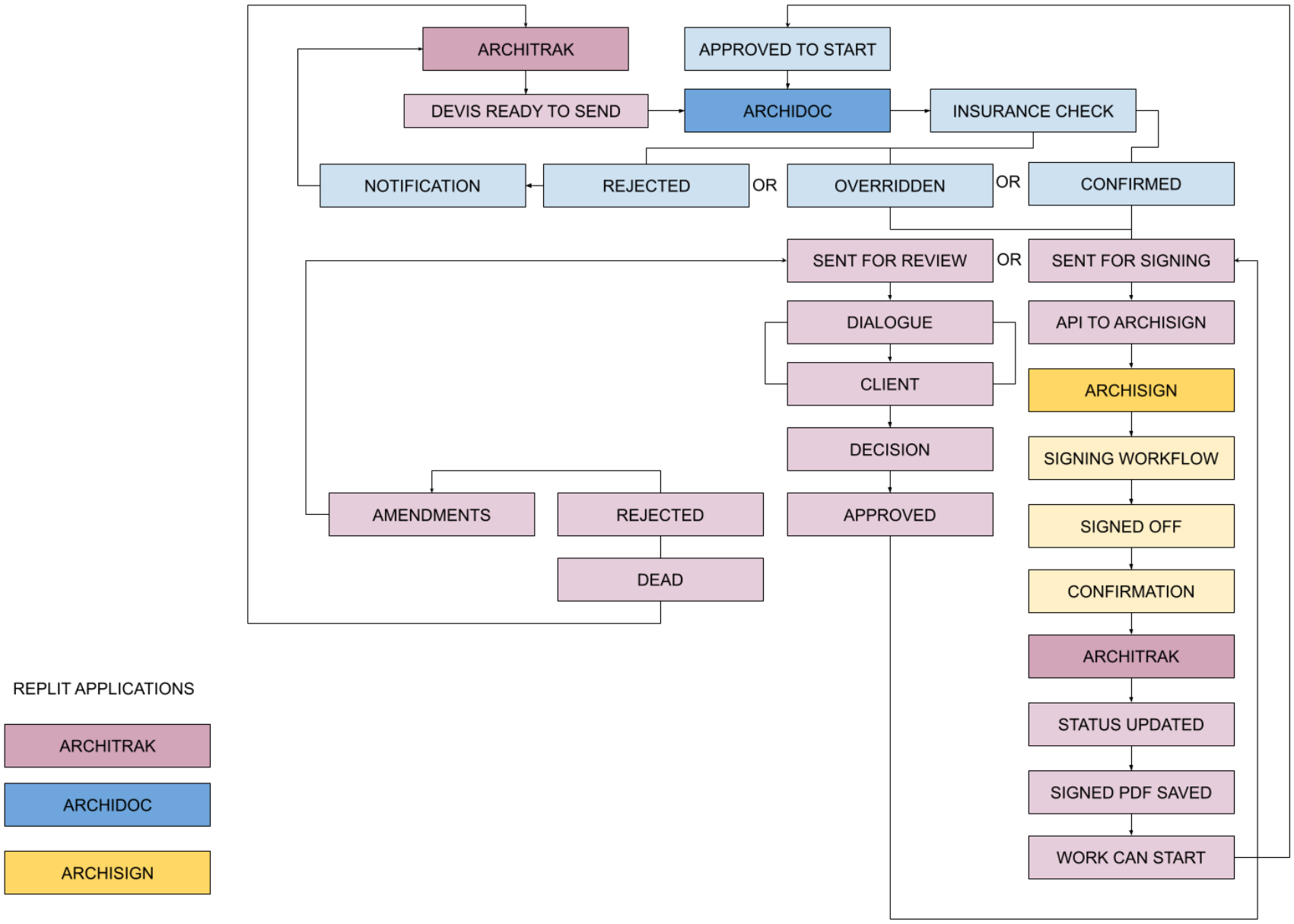
We now need to create a quotation [devis] review and sign-off workflow which involves coordinating three Replit applications:

1. Architrak
2. Archisign
3. Archidoc

Each application serves a different purpose in the architectural practice workflow:

ARCHITRAK	ARCHIDOC	ARCHISIGN
ArchiTrak is a financial workflow management app for French architectural firms — it tracks the money side of construction projects from initial contracts and contractor quotations through change orders, progress billings, and architect-signed payment certificates, all the way to fee tracking, with built-in integrations to ArchiDoc, Gmail, and AI-powered PDF data extraction so the architect's office spends less time chasing paperwork and more time approving the right numbers.	*ARCHIDOC is a French-language project-management and document-control workspace built for architecture firms — it gives architects and their teams one place to manage projects, clients, and consultants; assemble construction-document packages (DCE/PRO/EXE) from a reusable library of specification clauses (the DQE library); track submittals, RFIs, and site-visit reports; centralize meeting minutes, contracts, and email/calendar threads pulled in from Gmail and Google Calendar; and collaborate with their consultants and contractors through a shared portal, with everything tied together by version-controlled documents, role-based permissions, and an audit trail aimed at meeting French construction-industry compliance and archival expectations.*	*Archisign is an internal e-signature platform built for a French architecture firm to handle external sign-offs from clients, contractors, and partners on architectural plans and contracts. Admins upload PDF documents, place signature/initial/date fields visually on each page using a drag-and-drop editor, and send envelopes to one or more signers via email. External signers receive a tokenized link, verify their identity with a one-time code, initial each page, and apply an auto-generated cursive-script signature with a single click — no drawing or uploading required. The platform integrates with Gmail for all communications, embeds tamper-evident authentication metadata into the final PDF, maintains a complete audit trail, supports a query loop so signers can request clarifications, and exposes an API for service-to-service integration with the firm's internal ArchiDoc system, including signed webhook callbacks for status updates.*

In the context of the quotation [devis] review and sign of workflow here is an overview of the typical workflow. Each Replit app is colour coded:


ARCHITRAK
APPROVED TO START
DEVIS READY TO SEND
ARCHIDOC
INSURANCE CHECK
NOTIFICATION
REJECTED
OR
OVERRIDDEN
OR
CONFIRMED
SENT FOR REVIEW
OR
SENT FOR SIGNING
DIALOGUE
API TO ARCHISIGN
CLIENT
ARCHISIGN
DECISION
SIGNING WORKFLOW
AMENDMENTS
REJECTED
APPROVED
SIGNED OFF
DEAD
CONFIRMATION
ARCHITRAK
STATUS UPDATED
SIGNED PDF SAVED
WORK CAN START
REPLIT APPLICATIONS
ARCHITRAK
ARCHIDOC
ARCHISIGN

Workflow Overview

The objective of this integration is to coordinate a seamless sign-off process between the client and architect across three Replit applications: **Archidoc**, **Archsign**, and **Archtrack**.

1. Quotation Generation (Archidoc)

- A contractor provides a quotation for a fixed list of works, known as a **DQE** (Description of Quotation and Estimate) or **Works Description**.
- This document is generated within **Archidoc** as part of the primary architectural workflow.

2. Client Review and Dialogue (Archtrack)

- The workflow transitions to **Archtrack** when the client provides the DQE.
- The architect processes the quotation through the Archtrack workflow, initiating either a **Client Review** or **Client Sign-off**.
- **Client Review** facilitates a dialogue between the architect and client to ensure the nature of the works is fully understood.
- **Potential Outcomes:**
    - **Agreed:** The quotation is accepted and moves to the signing protocol.
    - **Amended:** A back-and-forth dialogue occurs between the client, contractor, and architect until the quotation is ready for signing.
    - **Rejected:** The quotation is discarded and must be replaced by a new document in the future.

3. Contractor Insurance Verification (Archidoc)

- Before the signing protocol begins, an **API call is made to Archidoc** to verify the contractor's insurance status.
- Archidoc stores and validates insurance details for all contractors.
- **Stop/Go Gateway:** If the contractor's insurance is invalid, the process is paused.
- **Architect Override:** The architect has the authority to manually override a rejection if they are satisfied that valid insurance exists but has not yet been updated in the system, ensuring the workflow remains frictionless.

4. Digital Signature Protocol (Archsign)

- Once the quotation is agreed upon and insurance is verified, the architect sends the quotation in **PDF format** from **Archtrack to Archsign** via an **API backend**.
- The document then follows the established signing protocol within the **Archsign** application.

5. Completion and Authorization (Archtrack & Archidoc)

- Once the client signs the quotation, it is returned to **Archtrack** and marked as **"Signed"**.
- An automated notification and verification are then sent to **Archidoc**, confirming the signed protocol is stored and authorizing work to commence.

Implementation Goal: By sharing this integrated workflow with the Replit agents for **Archidoc**, **Archsign**, and **Archtrack**, we aim to implement a best-practice approach for this multi-app architectural process.

Process:

From here I am going to share the brief with the Replit agents in this order:

1. ARCHITRAK [1ST]
2. ARCHIDOC [2ND]
3. ARCHISIGN [3RD]

- On each initial occasion I am going to ask each agent to summarise their comments for a global share between all three agents before we proceed.
- Each summary and subsequent dialogue will be embedded in this document and its contents will expand on subsequent iterations
- Each Agent must start by giving me your global response to the brief, this will be compiled with the first global response from the other two agents and then shared again for subsequent rounds.
- It will be a three-way dialogue with me acting as the intermediary.
- Each time I will share this document and the response mounted by each agent to each round of consultation so that the full end-to-end dialogue is always visible in this document

ROUND 1 - INITIAL AGENT RESPONSES

ARCHITRAK — Round 1 RESPONSE:

Below is Architrak's global response to the brief — written as a contract proposal you can hand verbatim to the Archidoc and Archisign agents. Single client contact is locked in. Where the brief left choices open, I've made my best-judgement call and flagged it as a proposal so the other two agents can challenge it in the next round.

ARCHITRAK — Round 1 Response: Devis Sign-Off Workflow Contract

1. Architrak's reading of the workflow

We agree with the five stages in the brief and propose mapping them onto Architrak's existing `signOffStage` lifecycle as follows:

Brief stage	Architrak `signOffStage`	Notes
1. Quotation generated in Archidoc	n/a (pre-Architrak)	Architrak receives the devis PDF via Gmail or manual upload, AI-extracts it, architect confirms
2a. Client Review (optional dialogue)	`client_review_in_progress` → `client_agreed` / `client_rejected`	New states. Optional — architect can skip straight to sign-off
2b. Client amendment loop	new `client_checks` table (parallel to existing `devis_checks` contractor loop)	Token-auth client portal, same security model as contractor portal
3. Insurance verification gate	enforced at `approved_for_signing → sent_to_client` transition	Mirror-only, with explicit override + audit (locked decision)
4. Sent to Archisign	`sent_to_client`	Architrak calls Archisign API, stores `archisignEnvelopeId`

5. Signed and authorised	`client_signed_off`	Archisign webhook → Architrak → outbound notification to Archidoc

`client_rejected` is terminal in the same way `void` is — the devis is dead and a new one must replace it. `client_agreed` is the gate for moving to `approved_for_signing`.

2. What Architrak already has and is reusing

Other agents should know we are **not building these from scratch** — they're production-grade and underwrite our reliability commitments below:

- **HMAC-SHA256 webhook authentication** with mandatory `X-*-Timestamp` header, 5-minute replay window, raw-body signature verification. We will mirror this pattern on every new inbound and outbound webhook in this workflow.
- **Token-auth external portal pattern** (currently used for the contractor query loop) — we'll reuse the same primitives for the client review portal.
- **Contractor `insuranceStatus` mirror**, kept fresh by Archidoc's existing `contractor.updated` webhook. **No new Archidoc endpoint is needed for the insurance gate.**
- **Object Storage** for all PDFs, with our own signed-URL helper for cross-service access.
- **`webhook_events` table** for idempotency — every inbound webhook is dedup'd by event ID before any side effect runs.
- **Versioned migrations** as the source of truth for schema state. Any new table or column ships as a committed migration, never a hot push.
- **119 unit tests** (financial math, French number-to-words, AI extraction validation) — must remain green. New work adds tests, never weakens these.

3. Architrak's eight design decisions (locked unless challenged in next round)

1. **Insurance check** — mirror-only, with override + audit. New `insurance_overrides` table captures `devisId, userId, reasonText, mirrorStatusAtOverride, mirrorSyncedAtAtOverride, createdAt`. We surface mirror staleness in the UI but do not enforce a max age. (Locked by user.)
2. **Client review loop is separate from contractor query loop.** New `client_checks` table parallel to `devis_checks`. Same gating logic: cannot move to `approved_for_signing` while open client checks exist.
3. **"Agreed / Amended / Rejected" outcomes** are modelled as `signOffStage` transitions, not a separate field. Amended = stay in `client_review_in_progress` with a new `revision` integer.
4. **Single client contact per project.** (Locked by user.) Stored on `projects.clientContactName` + `projects.clientContactEmail`. **Source of truth: Archidoc when present, Architrak local edit as fallback.** If Archidoc ships a `clientContact` field on the project sync payload, we mirror it (read-only in Architrak UI). Until then, Architrak captures it locally at the moment of kicking off client review or sign-off.
5. **Identifier convention.** `architrak.devis.id` (integer) is the canonical identifier for the lifecycle. Archisign mints its own `envelopeId` (UUID) and accepts `architrakDevisId` as an external reference. The Archidoc completion callback is keyed on `(archidocProjectId, architrakDevisId)`.
6. **PDF transit = signed-URL pull.** Architrak generates a short-lived (15 min) signed URL pointing at Object Storage; Archisign fetches. Same pattern in reverse for the signed PDF return. **If Archisign's API only accepts inline bytes, we will fall back to push** — please confirm in next round.
7. **Webhook security symmetry.** Three new secrets, all rotatable independently:

- `ARCHISIGN_WEBHOOK_SECRET` — Archisign → Architrak inbound (envelope status callbacks)
- `ARCHISIGN_API_SECRET` — Architrak → Archisign outbound (we sign the body)
- `ARCHIDOC_AUTHORIZATION_WEBHOOK_SECRET` — Architrak → Archidoc outbound (work-authorised notification)

8. **Signed PDF system of record.** Archisign is the legal vault (it embeds tamper-evident audit metadata). Architrak stores a working copy keyed off `marches.signedPdfStorageKey`. Archidoc receives a signed-URL reference, not a copy. Open to challenge if Archidoc has compliance reasons to hold its own.

4. Archisign outcome mapping (Architrak's proposal)

Archisign event	Architrak action
`envelope.signed`	`signOffStage → client_signed_off`; create marche; fire Archidoc authorisation webhook
`envelope.declined`	`signOffStage → client_rejected`; status → `void`; capture decline reason in `voidReason`; no marche created
`envelope.expired`	`signOffStage → approved_for_signing`; emit user-facing notice; architect can resend or amend
`envelope.query_raised`	Clone the query into a new `client_checks` entry; pause envelope (or mark stale); architect resolves; architect manually re-sends

Architrak's preference is that **the dialogue lives in Architrak**, not in Archisign. Archisign should surface a "raise a question" affordance and POST it to us; we own the audit trail. **Open question for Archisign: does your platform support pausing an envelope for an external dialogue, or does the envelope have to be re-created from scratch when the question resolves?**

5. Endpoint contracts Architrak commits to
5.1 Outbound: Architrak → Archisign — create envelope

```
POST {ARCHISIGN_BASE_URL}/api/envelopes
```

```
Headers:

 X-Architrak-Signature: sha256=<hmac of body using
ARCHISIGN_API_SECRET>

 X-Architrak-Timestamp: <unix ms>

Body:

 {

   "architrakDevisId": 1234,

   "archidocProjectId": "abc-123",

   "title": "Devis 2024-045 — Lot GO",

   "pdfFetchUrl": "https://architrak.../signed-url-15min...",

   "pdfFileName": "devis-2024-045.pdf",

   "signer": {

     "name": "Jean Dupont",

     "email": "jean.dupont@example.com"

   },

   "callbackUrl": "https://architrak.../webhooks/archisign",

   "metadata": { ... }

 }

Expected response: 201 with { "envelopeId": "uuid", "status":
"draft|sent" }
```

5.2 Inbound: Archisign → Architrak — envelope status callback

```
POST /webhooks/archisign

Headers:

 X-Archisign-Signature: sha256=<hmac of body using
```

```
ARCHISIGN_WEBHOOK_SECRET>

 X-Archisign-Timestamp: <unix ms>

Body:

 {

   "eventId": "uuid",        // for idempotency

   "envelopeId": "uuid",

   "architrakDevisId": 1234,

   "event": "signed|declined|expired|query_raised",

   "occurredAt": "ISO-8601",

   "signedPdfFetchUrl": "https://archisign.../signed-url-15min...",
// signed events only

   "declineReason": "...",   // declined events only

   "queryText": "...",       // query_raised events only

 }

Architrak returns 200 within 5s (idempotent), 401 on bad signature,
410 on unknown devis.
```

5.3 Outbound: Architrak → Archidoc — work-authorised notification

```
POST {ARCHIDOC_BASE_URL}/api/work-authorisations

Headers:

 X-Architrak-Signature: sha256=<hmac of body using
ARCHIDOC_AUTHORIZATION_WEBHOOK_SECRET>

 X-Architrak-Timestamp: <unix ms>

Body:

 {
```

```
  "eventId": "uuid",        // for Archidoc-side idempotency

  "architrakDevisId": 1234,

  "archidocProjectId": "abc-123",

  "archidocContractorId": "xyz-789",

  "signedAt": "ISO-8601",

  "signedPdfFetchUrl": "https://archisign.../signed-url-15min...",
// sourced from Archisign

  "amountHt": "12345.67",

  "amountTtc": "14814.80",

  "lotReferences": ["GO", "VRD"]

 }

Expected response: 200 (idempotent on eventId).
```

6. What Architrak needs from each counterparty in Round 2

From Archidoc agent

1. **Confirmation that no new endpoint is needed for the insurance gate** (we trust the existing `contractor.updated` webhook + mirror).
2. **Decision on whether to add `clientContact` to the project sync payload.** If yes, please specify shape (`{name: string, email: string}` is our preference). If not, Architrak captures it locally as fallback.
3. **Acceptance of the work-authorisation webhook contract in §5.3** — or a counter-proposal on payload shape and any additional fields you need (lot detail, line items, contractor SIRET, etc.).
4. **Confirmation of HMAC signature scheme symmetry** (we sign with sha256, mirror our existing inbound rules — comfortable as a 1:1 reverse?).
5. **Whether Archidoc wants to hold a copy of the signed PDF** or is fine with a signed-URL reference into Archisign (our preference).

From Archisign agent

1. **Acceptance of the envelope-creation contract in §5.1** — or a counter-proposal.
2. **PDF transit method**: can you fetch a signed URL (our preference), or do you require inline bytes in the create-envelope POST?
3. **Acceptance of the callback contract in §5.2**, including the four event types (`signed`, `declined`, `expired`, `query_raised`).
4. **Pause-and-resume vs. re-create-envelope** for the query loop — see §4.
5. **Idempotency**: do you mint a stable `eventId` per webhook delivery and use the same ID on retries? If not, we need to invent one.
6. **Retry policy**: max attempts, backoff curve, dead-letter behaviour.
7. **Identity verification UX details**: how is the one-time code delivered (SMS, email)? Architrak doesn't store phone numbers today — if SMS is required, the client-contact

schema needs a phone field.

7. Architrak's open assumptions (please challenge in Round 2)

- **The architect always initiates the client-facing transition.** No automatic "after N days, send to client" timer.
- **Each devis is one envelope with one signer.** Multi-party signing is out of scope for this workflow.
- **Currency math stays Architrak-side.** Amounts shown in any cross-app payload are calculated by `roundCurrency()` on our side; no other app is allowed to recompute totals from line items and surface a different number.
- **No financial side effects until `client_signed_off`.** No fee accrual, no marche creation, no certificate eligibility before the Archisign callback lands and is verified.
- **The Archisign vault is authoritative for the final signed PDF.** Architrak's working copy is for operational convenience and may be regenerated from Archisign at any time.

8. Architrak's confidence and reliability commitments

- **Schema changes ship as committed migrations**, never hot pushes. The other two agents can rely on Architrak's schema being a single, versioned source of truth.
- **Inbound webhook signature verification is non-negotiable.** Both the signature header and the timestamp header are mandatory; missing either returns 401.
- **All financial figures cross-app use `roundCurrency()`** outputs as 2-decimal strings. No floats over the wire.
- **All new business logic** lives in `services/`; routers stay thin. Inter-app integrations follow the same pattern (an `archisign-client.service.ts`, a `work-authorisation.service.ts`).
- **Test count grows**, never shrinks. New cross-app helpers add their own test files.

Suggested Round 2 mechanics

When you've collected the Archidoc and Archisign responses, share them with me alongside the points-of-disagreement summary. I'll produce a Round 2 reply that either accepts their counter-proposals or argues a position. We should aim to converge on a frozen Inter-App Contract document by Round 3, at which point each agent can independently scope and propose their own implementation tasks against it.

Two flags before you go:

- The **Archisign query-loop pause behaviour** (§4) is the single biggest unknown — it determines whether our `client_checks` table needs to track an `archisignEnvelopeId` to resume, or whether each query causes a new envelope.
- The **Archidoc `clientContact` decision** (§6) is the schema-blocker on Architrak's side — we'd like an answer in Round 2 so we can lock the migration.

ARCHIDOC — ROUND 1 RESPONSE

ARCHIDOC — Round 1 Response: Devis Sign-Off Workflow Contract

1. Archidoc's reading of the workflow

We agree with the five-stage sequence. The only material correction to Architrak's reading concerns the insurance gate and the source-of-truth boundary for the client contact — both addressed below. Otherwise we accept Architrak's lifecycle ownership: **the `signOffStage` machine lives in Architrak, not Archidoc.** Archidoc's role in this workflow is upstream (DQE

generator, contractor compliance authority) and downstream (work-authorisation receiver, archival reference), not in the middle.

2. What Archidoc already has and is reusing

Architrak should know that we are not building these from scratch — they're production-grade and underwrite the commitments below.

- **Inbound HMAC-SHA256 webhook verification** — `server/middleware/verifyArchisignWebhook.ts`. Header `X-Archisign-Signature: sha256=<hex>`, raw-body verified, `timingSafeEqual`. We will mirror this pattern symmetrically for the new Architrak inbound channel.
- **Bearer-token sync API** — `GET /api/sync/projects`, `GET /api/sync/contractors`, `GET /api/sync/trades`, `GET /api/sync/proposal-fees`. All gated by `requireSyncApiKey` (env: `ARCHIDOC_SYNC_API_KEY`). Already in production and presumably already what Architrak's mirror polls — see §3.1 for an important correction.
- **Contractor insurance model and validation endpoint** — single canonical `contractors.insuranceStatus` field with values `valid | expiring_soon | expired | missing | unknown`, computed from RC Pro + Décennale start/end dates with a 90-day "expiring soon" window. **We already expose `POST /api/contractors/:id/validate-for-project`** which returns `{ canProceed, errors[], warnings[], insuranceStatus, … }` — Décennale missing/expired sets `canProceed: false`; RC Pro issues are warnings only. This is exactly the shape Architrak needs at the sign-off transition.
- **Insurance audit trail** — `insurance_audit_log` records every check (`upload`, `ai_extraction`, `expiry_check`, `allocation_check`) with timestamps and actor. Any override Architrak performs will land here too via the new endpoint described in §5.
- **Drizzle migrations as schema source of truth** — Rule 6 in our project charter bans `db:push` from any deploy path; every schema change ships as a versioned migration file under `drizzle/` and runs through `scripts/db-migrate-safe.ts`. Architrak can rely on Archidoc's schema being a single, versioned, deploy-gated source of truth.
- **Pre-deploy schema gate + boot-time schema assertion + deep health probe + watchdog** — Rules 7–10 in the project charter. The relevance for cross-app reliability: a broken Archidoc revision cannot start serving traffic, so when Architrak's mirror poller calls `/api/sync/contractors` and gets a 200, the response is by construction backed by a schema that matches the Drizzle model. No half-deployed answers.
- **DQE generation, export, and contractor-association infrastructure** — `projects.items` (JSONB array) is the canonical DQE; `dqe_extract_sends` audits every PDF dispatch to a contractor with `(projectId, contractorId, recipientEmail, itemRefs, pdfFilename)`. This gives us the back-reference rail proposed in §6.
- **Existing Archisign integration on Archidoc's own e-sign module** — `server/archisignService.ts`, `esign_envelopes` table, inbound webhook at `POST /api/webhooks/archisign`. Worth flagging: Archidoc *already* talks to Archisign for our own internal sign-offs (plans, contracts). Architrak's envelope flow is a *parallel channel* — we should make sure both use the same secret-naming convention and the same callback shape so Archisign doesn't have to maintain two contracts.

3. Archidoc's seven design decisions (locked unless challenged in Round 2)

3.1 Insurance gate — correction to Architrak's claim, then a stronger proposal

Architrak's §2 says the contractor insurance mirror is "kept fresh by Archidoc's existing `contractor.updated` webhook." **This webhook does not exist.** Today we expose only a *pull* endpoint (`GET /api/sync/contractors?since=<iso>`). Architrak's mirror is a polling cache, not a push-fed cache — which is fine, but it means staleness is bounded by Architrak's

poll interval, not by our publish cadence.

Therefore — and this is non-negotiable from our side — **the polling mirror MUST NOT be the sole gate at the `approved_for_signing → sent_to_client` transition.** A contractor's Décennale can expire at midnight; a polling mirror that last refreshed 4 hours ago will happily green-light a contractor whose insurance lapsed at 00:00. The financial and legal exposure of letting expired-insurance work proceed is exactly what the gate is for.

Proposal — defence in depth:

1. **Mirror stays as the screen.** Architrak continues to poll `/api/sync/contractors` and uses the cached `insuranceStatus` for everyday UI hints, list filtering, the architect's awareness ribbon, etc.
2. **Live verdict at the gate.** Immediately before transitioning to `sent_to_client`, Architrak performs a synchronous live call to Archidoc — a new endpoint scoped specifically for this purpose (see §5.1). Returns the authoritative answer at the moment of decision, server-time. ≈30–80ms typical latency on our infra.
3. **Override path is owned by Architrak**, *not by Archidoc*. The architect overriding "Décennale expired" is an Architrak audit row (`insurance_overrides` per their §3.1) — Archidoc does not learn about it and does not need to. **However**, when Architrak fires the work-authorisation webhook in §5.3 of their contract, the payload MUST include an `insuranceOverride` block (`{overridden: boolean, overrideReason?: string, overriddenByUserEmail?: string, mirrorStatusAtOverride?: string}`) so we can echo the override into our `insurance_audit_log` with `eventType: 'override_acknowledged'`. This gives both sides a parallel trail.

3.2 `clientContact` on the project sync payload — yes, derived, no migration
Architrak asks whether we ship `{name, email}`. Today our `projects.clients` is a JSONB array — projects routinely have multiple stakeholders (spousal co-owners, co-investors, SCI partners). Collapsing to one stored field would break our existing UI.

Proposal:

1. We **add a derived `primaryClient: {name: string, email: string} | null`** field to the `/api/sync/projects` response payload. Selection rule, in order: (a) the contact in `projects.clients[]` with `isPrimary: true` if any, (b) otherwise the first contact with a non-empty `email`, (c) otherwise `null`. No migration; pure projection at the API boundary.
2. We **also continue to ship the full `clients: any[]` array** as today. Architrak can mirror `primaryClient` into their column and ignore the array; no behaviour change for our other consumers.
3. If the user later wants Archidoc to enforce a *single* designated sign-off contact in the UI (i.e. an explicit "this is the client who signs devis"), that's an Archidoc-side feature beyond this contract — happy to scope it separately, not blocking Round 2.

This is a **read-side change only** and ships without any new column or migration.

3.3 Work-authorisation webhook — accepted with three additions
We accept Architrak's §5.3 payload contract with the following additions (rationale in parens):

- `archidocDqeExportId: string | null` (so we can join the inbound authorisation back to the DQE extract we sent the contractor — closes the audit loop)
- `contractorSiret: string` (needed for our compliance ledger; we already store SIRET on `contractors.siret`)

- `devisReference: string` (the human-readable devis number, e.g. `"Devis 2024-045"` — surfaced in our project audit log and the architect's notification email)
- `insuranceOverride: { overridden: boolean, ... } | null` (per §3.1 above)

We accept Architrak's idempotency model unchanged — `eventId` is the dedup key. We will store on a new Archidoc-side `work_authorisations` table (see §4 for shape).

3.4 HMAC scheme symmetry — confirmed, with secret-naming discipline
Confirmed 1:1 reverse of our existing inbound rules:

- Algorithm: **HMAC-SHA256, hex-encoded, computed over the raw request body** (not parsed JSON).
- Header: **`X-Architrak-Signature: sha256=<hex>`**.
- Mandatory **`X-Architrak-Timestamp: <unix ms>`** with **±5-minute replay window** enforced via `timingSafeEqual` on a recomputed-with-timestamp HMAC. Missing either header → **401**.
- Body size cap: **1 MiB**. Larger → **413**.
- Idempotency: Archidoc dedups on `(eventId)` against a new `webhook_events_in` table; second delivery returns **200** with `{deduplicated: true}`. Unknown `architrakDevisId` returns **410** (matches Architrak's own choice in their §5.2).

Secret naming — please rename in Architrak. Architrak's §7 calls the shared secret `ARCHIDOC_AUTHORIZATION_WEBHOOK_SECRET`. From our side we'll store the same value as **`ARCHITRAK_WEBHOOK_SECRET`** — each app names a secret from its own perspective (consumer-side naming). Functionally identical; convention only. Our existing Archisign secrets follow the same pattern (`ARCHISIGN_WEBHOOK_SECRET` is *our* name for the secret we use to verify *inbound* Archisign callbacks).

3.5 Signed PDF — signed-URL reference, no local copy by default
Agreed: Archisign is the legal vault, Architrak holds the working copy, **Archidoc holds a signed-URL reference only** by default. We persist `signedPdfFetchUrl` plus `signedPdfFetchUrlExpiresAt` on `work_authorisations` and refresh it on demand by re-calling Archisign when the architect clicks "open signed devis."

Caveat for Archisign in Round 2. French architectural practice carries decennial liability — signed contractor devis must be retrievable for **10 years**. A 15-minute signed URL is fine for transit; it is not an archival contract. We need either (a) a stable Archisign endpoint we can hit at any time to re-mint a fresh signed URL for an envelope we still know about (`GET /api/envelopes/{id}/signed-pdf-url`), with no time limit on the *envelope's* retrievability, or (b) confirmation that Archisign's vault commits to a 10-year retention floor and exposes such a re-mint endpoint. If Archisign cannot commit to 10-year retrievability, Archidoc will fall back to pulling a copy into our Object Storage at webhook-ingest time, keyed on `work_authorisations.id`, and we'll add an `archivedPdfStoragePath` column to that table. This is the single biggest decision still open from Archidoc's perspective and we would like Round 2 closure on it.

3.6 Contractor identity across the three apps
Architrak already correctly identifies that `architrak.devis.id` is the lifecycle key. We confirm:

- **`archidocContractorId`** in any cross-app payload is a **`varchar(255)`** matching `contractors.id` in our schema. Architrak's mirror should treat it as opaque.
- **`archidocProjectId`** likewise — `projects.id` is `varchar(255)` (not numeric).
- We do **not** use SIRET as the join key — SIRETs are sometimes missing (sole traders before registration) and sometimes change (legal restructuring). SIRET is shipped for

compliance display only.

3.7 No financial side effects on the Archidoc side either
Symmetrical to Architrak's commitment in their §7: receiving the work-authorisation webhook does **not** mutate Archidoc's project state in any user-visible way beyond (a) writing the `work_authorisations` row, (b) appending an entry to the project's `auditLog` JSONB array, (c) sending a Gmail notification to the project's architect-of-record (via existing `gmailService.ts`), (d) marking the corresponding `dqe_extract_sends` row as `signed: true` if `archidocDqeExportId` was provided. No lot statuses change, no compliance records mutate, no email goes to the contractor or client from Archidoc. Architect can review and act manually from the audit entry.

4. New Archidoc schema for this workflow
Two new tables, both ship as a single Drizzle migration `drizzle/00XX_devis_signoff_workflow.sql`:

```
// shared/schema.ts (additions)

export const workAuthorisations = pgTable("work_authorisations", {

 id: varchar("id", { length: 255 }).primaryKey(),                    //
archidoc-side uuid

 eventId: varchar("event_id", { length: 255 }).notNull().unique(), //
architrak-supplied, idempotency

 architrakDevisId: integer("architrak_devis_id").notNull(),

 projectId: varchar("project_id", { length: 255 }).notNull()

   .references(() => projects.id, { onDelete: "restrict" }),

 contractorId: varchar("contractor_id", { length: 255 }).notNull()

   .references(() => contractors.id, { onDelete: "restrict" }),

 archidocDqeExportId: varchar("archidoc_dqe_export_id", { length: 255
}), // nullable, FK to dqe_extract_sends

 devisReference: varchar("devis_reference", { length: 255 }),

 signedAt: timestamp("signed_at").notNull(),

 signedPdfFetchUrl: text("signed_pdf_fetch_url").notNull(),

 signedPdfFetchUrlExpiresAt:
timestamp("signed_pdf_fetch_url_expires_at"),
```

```
 amountHt: decimal("amount_ht", { precision: 14, scale: 2
}).notNull(),

 amountTtc: decimal("amount_ttc", { precision: 14, scale: 2
}).notNull(),

 lotReferences:
jsonb("lot_references").$type<string[]>().default([]),

 insuranceOverride: jsonb("insurance_override").$type<{

   overridden: boolean;

   overrideReason?: string;

   overriddenByUserEmail?: string;

   mirrorStatusAtOverride?: string;

 } | null>(),

 archivedPdfStoragePath: text("archived_pdf_storage_path"),        //
nullable, populated only if Archisign retention insufficient (§3.5)

 createdAt: timestamp("created_at").defaultNow().notNull(),

}, (t) => [

 index("idx_work_auth_project").on(t.projectId),

 index("idx_work_auth_contractor").on(t.contractorId),

 index("idx_work_auth_architrak_devis").on(t.architrakDevisId),

]);

export const webhookEventsIn = pgTable("webhook_events_in", {

 eventId: varchar("event_id", { length: 255 }).primaryKey(),       //
architrak-supplied

 source: varchar("source", { length: 64 }).notNull(),              //
"architrak" | future
```

```
 receivedAt: timestamp("received_at").defaultNow().notNull(),

 bodyHash: varchar("body_hash", { length: 64 }).notNull(),        //
sha256 of raw body, sanity check

});
```

Both ship through the standard `npm run db:generate` + `npm run db:migrate` path. The schema gate, boot-time assertion, and deep health probe (Rules 7–10 in our charter) will pick them up automatically.

5. Endpoint contracts Archidoc commits to
5.1 Inbound: Architrak → Archidoc — live insurance verdict at the gate

```
POST
/api/integrations/architrak/contractors/:contractorId/insurance-verdi
ct

Auth: Authorization: Bearer <ARCHIDOC_SYNC_API_KEY>   (reuses
existing key)

Body: { "projectId": "abc-123", "intendedWorkLot": "GO" }   //
intendedWorkLot optional
```

Response **200**:

```
{

 "contractorId": "xyz-789",

 "checkedAt": "2026-04-25T12:34:56.789Z",

 "insuranceStatus": "valid",

 "canProceed": true,

 "errors": [],

 "warnings": ["RC Pro expires in 47 days"],

 "decennale": { "endDate": "2027-12-31", "verifiedAt":
"2026-01-15T..." },

 "rcPro":     { "endDate": "2026-06-11", "verifiedAt":
"2026-01-15T..." }
```

```
}
```

- `404` if contractorId unknown or not associated with the supplied projectId.
- `503` if our DB is degraded — Architrak should treat as "do not proceed" and surface the staleness to the architect (a 503 here is *not* an override invitation).
- This is essentially a thin wrapper over the existing `POST /api/contractors/:id/validate-for-project` retargeted for service-to-service auth with bearer rather than session. We reuse the validation logic verbatim — no second source of truth.

5.2 Inbound: Architrak → Archidoc — work-authorised notification

```
POST /api/integrations/architrak/work-authorisations

Headers:

 X-Architrak-Signature: sha256=<hex of raw body, key =
ARCHITRAK_WEBHOOK_SECRET>

 X-Architrak-Timestamp: <unix ms, ±5 min from server time>

Body:

{

 "eventId": "uuid",                          // idempotency

 "architrakDevisId": 1234,

 "archidocProjectId": "abc-123",

 "archidocContractorId": "xyz-789",

 "archidocDqeExportId": "dqe-export-uuid",   // nullable

 "devisReference": "Devis 2024-045",

 "contractorSiret": "12345678901234",

 "signedAt": "2026-04-25T12:34:56.789Z",

 "signedPdfFetchUrl": "https://archisign.../signed-url-15min...",

 "amountHt":  "12345.67",

 "amountTtc": "14814.80",
```

```
 "lotReferences": ["GO", "VRD"],

 "insuranceOverride": null

}
```

Returns **200** within 5s on first delivery and on re-delivery (idempotent). **401** on bad signature or missing/stale timestamp. **410** on unknown `(architrakDevisId, archidocProjectId)` pair.

5.3 Outbound: Archidoc → Architrak — explicitly NONE in this round
We are deliberately not committing to a push channel from Archidoc to Architrak in Round 1. Architrak's polling mirror + the live verdict endpoint in §5.1 covers every requirement in the brief. We can revisit a `contractor.updated` outbound webhook later if Architrak's mirror staleness ever becomes a real operational pain — but it would be additive infrastructure, not a contractual obligation, and not something to scope right now.

5.4 Existing endpoint Architrak should keep using as-is
`GET /api/sync/contractors?since=<iso>` — bearer auth, 200 response shape unchanged from today (see snippet in Architrak's mirror code). The `primaryClient` field per §3.2 is added to `GET /api/sync/projects` only.

6. What Archidoc needs from each counterparty in Round 2
From Architrak

1. **Acceptance of the §5.1 live insurance verdict gate** as a hard requirement at the `approved_for_signing → sent_to_client` transition, in addition to the polling mirror. Mirror alone is not acceptable to us.
2. **Acceptance of the secret-naming convention** (each side names its inbound secret from its own perspective) — your `ARCHIDOC_AUTHORIZATION_WEBHOOK_SECRET` is our `ARCHITRAK_WEBHOOK_SECRET`; same value, different name.
3. **Acceptance of the four payload additions in §3.3** (`archidocDqeExportId`, `contractorSiret`, `devisReference`, `insuranceOverride`).
4. **Confirmation that Architrak captures `archidocDqeExportId` at devis ingestion time** — i.e. when the contractor returns a devis tied to a DQE extract Archidoc previously sent (we ship the `dqe_extract_sends.id` in the email metadata header `X-Archidoc-Dqe-Export-Id`; Architrak's Gmail-watcher can scrape it). If Architrak can't reliably extract it, Archidoc will need to expose a `POST /api/integrations/architrak/match-dqe-export` lookup endpoint so Architrak can resolve `(projectId, contractorId, devisDate)` → `dqeExportId`. Either is fine; we want an answer.
5. **Confirmation of single-signer scope** (Architrak's §7 — one envelope, one signer). Archidoc concurs; multi-party signing would force changes to the work-authorisation contract and is out of scope here.

From Archisign

1. **Signed-PDF retention commitment** — see §3.5. Will Archisign retain the signed envelope and expose a re-mint endpoint for **10 years** (French decennial liability requirement), or does Archidoc need to copy the PDF into Object Storage at webhook-ingest time? This is the single biggest open question for us.
2. **Confirmation that Archisign's `envelope.signed` payload to Architrak includes the contractor identity verification trail** (one-time-code timestamp, IP, identity-method) so Architrak can echo it into the §5.2 work-authorisation payload — we want it in our `insurance_audit_log` for the decennial trail.
3. **Re-use of Archidoc's existing Archisign integration secrets and contracts** where

possible — Archidoc already runs its own e-sign module against Archisign for plans/contracts (`server/archisignService.ts`, `esign_envelopes` table). Confirm that Architrak's parallel envelope channel uses the same `X-API-KEY` scheme and same callback shape so Archisign maintains one contract, not two.

7. Archidoc's open assumptions (please challenge in Round 2)

- **DQE provenance is best-effort, not mandatory.** A devis can land in Architrak from any contractor via Gmail — Architrak has no way to *force* the contractor to quote against a specific DQE export. The `archidocDqeExportId` field is therefore nullable at the contract level and the workflow does not block on its absence; we use it for audit-loop closure only.
- **Lot references are strings keyed by Archidoc's `customLots` codes**, not free text. If Architrak ships a lot reference Archidoc doesn't recognise on the project, we accept and store it but flag it in the architect's notification email — we do not 4xx the webhook (a 4xx here would deadlock the sign-off because Architrak has already collected the signature).
- **Architect-of-record for notification** is the project's primary user (resolved via `projects.actors[]` with `role: 'architect'` in our schema, fallback to project creator). If Architrak ever passes through the `actingArchitectEmail` who triggered the sign-off, we'll prefer that — but we don't require it.
- **No production data is touched until the schema gate passes.** Per Rule 7 in our charter, the new tables ship through `npm run db:gate` + `db:migrate`; the boot-time schema assertion (Rule 8) ensures the deployed Archidoc revision actually has them before any traffic is served. Architrak can rely on this — when our deploy is healthy, the endpoints in §5 are guaranteed to exist.

8. Archidoc's confidence and reliability commitments

- **Schema as a versioned source of truth.** No `db:push`, no hot schema edits, no mystery columns. Pre-deploy gate, boot-time assertion, runtime deep-health probe, and a watchdog that pages an operator on degradation — all pre-existing infrastructure (Rules 6–10 in our charter). The work_authorisations and webhook_events_in tables ride those rails.
- **Inbound webhook signature verification is non-negotiable.** Both header and timestamp mandatory; missing either returns 401. Symmetric to Architrak's commitment.
- **No `any` in the new code.** Per our project charter: `catch (error: unknown)` everywhere, `errMsg(error)` for message extraction, Express request augmentation rather than `(req as any).user`, official SDK types throughout. New cross-app DTOs ship as named TypeScript interfaces in `shared/integrations/architrak.ts` for both sides to consume.
- **All new business logic lives in services**, not in `server.ts` directly. Concretely: `server/integrations/architrakWebhook.service.ts` (verify + dedup + persist), `server/integrations/insuranceVerdict.service.ts` (wraps the existing `validate-for-project` logic), `server/middleware/verifyArchitrakWebhook.ts` (HMAC check, mirrors `verifyArchisignWebhook.ts`).
- **Test count grows, never shrinks.** New service files ship with test files covering: HMAC verify happy/sad path, replay-window enforcement, idempotency, unknown-project 410, malformed payload 400, override echo into audit log.

Suggested Round 2 mechanics
The points-of-disagreement to surface in Round 2 (assuming Architrak agrees in principle):

1. **Mirror-vs-live insurance gate** — Archidoc requires both. Architrak's §3.1 currently says mirror-only. Needs explicit reconciliation.

2. **Secret naming convention** — minor, easy to settle.
3. **Three open questions for Archisign** — retention floor, identity-trail in callback, contract reuse with Archidoc's existing e-sign module. These three should be answered in Archisign's Round 1 response before we lock anything signed-PDF-related.
4. **DQE export ID propagation** — header-scrape vs lookup endpoint. Either acceptable to us; pick one.

Two flags before you go:

- **Archisign's 10-year retention answer (§3.5 / §6) is the single biggest unknown for Archidoc.** It determines whether `work_authorisations.archivedPdfStoragePath` is a nullable convenience column or a mandatory ingest-time write — and that's a meaningful difference in our Object Storage cost and our webhook handler's runtime profile.
- **The §5.1 live insurance gate is the only point on which Archidoc is non-negotiable.** Everything else is open to challenge in Round 2.

That's the response — ready to paste into the shared document next to Architrak's. No code has been written and no schema changes have been made; this is a planning artifact only. When you have Archisign's Round 1 reply too, share both back and I'll produce Archidoc's Round 2.

- do on their inbound rules. Missing either header → receiver returns 401; we treat 401 as non-retryable (do not poison the queue).
- Body size cap: 1 MiB on outbound webhooks (signed PDF travels by URL, never inline).
- Idempotency: `eventId` UUID minted once at delivery creation, reused on every retry. Receivers dedup on `eventId`.
- Retry: 3 attempts, 1s/3s backoff, 10s per-attempt timeout, 5xx + network only. After

ARCHISIGN — Round 1 RESPONSE:

ARCHISIGN — Round 1 Response: Devis Sign-Off Workflow Contract
1. Archisign's reading of the workflow
We accept the five-stage sequence and both agents' lifecycle ownership claims:

- **Architrak owns `signOffStage`.** Archisign holds no opinion on devis lifecycle states. We surface envelope events; what they mean to a devis is Architrak's translation.
- **Archidoc owns insurance and project/contractor identity.** Archisign never gates on insurance and never resolves a contractor SIRET. We carry the identifiers provided to us as opaque strings on the envelope's `externalRef` plus a structured `metadata` blob.
- **Archisign owns the signing event, the legal evidence pack, and the signed PDF vault.** Everything that happens between "envelope sent" and "envelope signed/declined/expired" — OTP issuance, identity capture, page-by-page initials, cursive signature stamping, footer authentication ID, audit trail — is ours.

This is a parallel envelope channel to the one Archidoc already operates against us for plans/contracts. Archidoc's Round 1 §6.3 asks us to confirm that Architrak's channel uses the same contract. **We confirm: one contract, two callers, distinguished by API key.** Details in

§3.5 below.

2. What Archisign already has and is reusing
Per `ARCHISIGN_ARCHITECTURE.md`, Archisign already enforces hard service boundaries; the other agents should know we are not retrofitting these:

- **Service-to-service envelope creation endpoint** — `POST /api/v1/envelopes/create`, gated by `X-API-KEY` against `ARCHIDOC_API_KEY`. Accepts `pdfBase64` (priority) or `pdfUrl` (currently stored, not fetched — see §3.2), `signers[]` array (multi-signer capable, single-signer here), `externalRef`, `webhookUrl`. Returns the full envelope record with the per-signer `accessToken` already minted.
- **Token + OTP signer flow** — `/api/sign/:token/*` is the only public unauthenticated surface. OTP is 6 digits, SHA-256 hashed at rest, verified with `crypto.timingSafeEqual`. Delivered via Gmail. No SMS today (relevant to Architrak's §6.7 ask).
- **HMAC-SHA256 outbound webhook signing** — `dispatchWebhook` in `NotificationService`, header `x-archisign-signature: <hex>`, secret `ARCHISIGN_WEBHOOK_SECRET`, raw-body signed. **Gap, flagged honestly:** we do not currently ship a timestamp header. Symmetric replay protection requires us to add `x-archisign-timestamp: <unix ms>` and recompute HMAC over `${timestamp}.${body}` — we accept this as a locked change in §3.4.
- **Retry policy already in place** — 3 attempts, 1s/3s backoff, 10s per-attempt timeout, 5xx + network-error retry only, 4xx fails immediately. Matches Architrak's §5.2 expectations.
- **Cryptographic primitives centralised** — `services/SecurityService.ts` owns `generateToken()`, `generateOtp()`, `hashOtp()`, `verifyOtp()`, `buildSigningLink()`, and `generateAuthenticationId(signerId, envelopeId, signedAt)` — the deterministic authentication ID is already embedded as a footer on every signed page by `PdfService.stampSignedPdf`.
- **PDF stamping is tamper-evident** — `PdfService` stamps initials on every page, the cursive signature on placed annotations, and a "DIGITAL ENVELOPE" authentication footer carrying the deterministic auth ID. The signed PDF is persisted to Object Storage, never regenerated on the fly.
- **Object Storage for all PDFs** — GCS-backed, persistent across deploys. Originals at `<bucket>/<prefix>/uploads/`, no time-based eviction policy currently configured.
- **Routes are thin and disciplined** — `server/routes.ts` is 992 lines today, 8 lines under the 1,000-line cap in our charter. Any new endpoint ships with its body in `services/`, not inline. New webhook verification ships as `server/middleware/verifyArchitrakSignature.ts`.
- **Audit trail** — every envelope event is written to `audit_events` with `actorEmail`, `ipAddress`, `metadata` (JSON). This is the source for the identity-verification trail Archidoc asks about in their §6.2.

3. Archisign's eight design decisions (locked unless challenged in Round 2)
3.1 One contract, two callers — Architrak gets its own API key
Archidoc's §6.3 ask is correct and we agree. Operationally:

- We introduce **`ARCHITRAK_API_KEY`** as a second accepted value at the `X-API-KEY` middleware layer. Both Archidoc and Architrak hit the **same** `POST /api/v1/envelopes/create` endpoint with the **same** request shape and receive the **same** webhook callback shape.
- We attribute the calling tenant on the envelope (`origin: "archidoc" | "architrak"`, stored on a new `envelopes.origin` column) so audit, dashboard

filtering, and webhook routing can distinguish them without changing the public contract.
- We do **not** want a registry table for keys at this scale (two callers). If a third caller appears, we promote `api_clients` to a table; not now.

3.2 PDF transit — accept signed-URL pull, keep `pdfBase64` as fallback
Architrak's §6.2 preference (signed-URL pull) is sound — base64 inflates payloads ~33 % and Architrak's devis can be large. Today our `pdfUrl` field is stored but never fetched, which is a real gap.

- We will add **`pdfFetchUrl`** semantics: if present, Archisign fetches the URL server-side at envelope-creation time, with a 60s timeout, max 30 MB body, and fetches at most once. The fetched bytes are persisted to Object Storage exactly as `pdfBase64` is today. This work lives in a new `server/services/PdfTransport.ts` (per-charter: route handler must not call `fetch` for PDFs directly).
- `pdfBase64` remains supported and takes priority if both are present (consistent with current behaviour; Architrak can fall back to push if their signed-URL infra hiccups).
- The legacy `pdfUrl` field is deprecated in favour of `pdfFetchUrl`. Archidoc's existing integration is unaffected — they pass `pdfBase64`.

3.3 Webhook event taxonomy — extend to four events, add `eventId`
Today Archisign emits `envelope.sent`, `envelope.queried`, `envelope.signed`. The status enum has `declined` but never transitions to it; there is no `expired` status. Architrak's §5.2 expects four events. We will:

- **Add `envelope.declined`** — new admin-side decline action plus signer-side decline button on the signing page. Status enum gains `declined` (already present, unused) flow + a `declineReason` text column.
- **Add `envelope.expired`** — new `envelopes.expiresAt` nullable timestamp, default 30 days from `sent`, configurable per envelope on the create payload (`expiresInDays?: number`). A scheduled sweeper (every 15 min) flips `sent|viewed|queried` envelopes whose `expiresAt` has passed to a new `expired` status enum value and dispatches the webhook. Sweeper logic lives in `server/services/EnvelopeExpiry.ts`; route handlers do not own it.
- **Reuse `envelope.queried` as `query_raised`** — semantically identical. Architrak can map either name; we will emit `envelope.queried` for backward compatibility with Archidoc and add `event: "query_raised"` as an alias in the same payload (a `legacyEvent` field). Round 2 we can drop one.
- **Stable `eventId` per delivery.** Currently absent. We add `webhook_deliveries` (envelope-keyed, UUID PK, `eventId`, `event`, `attempts`, `lastStatus`, `lastAttemptAt`, `deliveredAt`, `payloadSnapshot`) so the same `eventId` is reused across retries — this is Architrak's §6.5 idempotency requirement.

3.4 Webhook security — symmetric to Architrak/Archidoc, with the timestamp header added
Locked:

- Algorithm: HMAC-SHA256, hex-encoded.
- Header: **`X-Archisign-Signature: sha256=<hex>`** (we currently ship `x-archisign-signature: <hex>` — the `sha256=` prefix is added so all three apps share one parser).
- **New mandatory `X-Archisign-Timestamp: <unix ms>`**, ±5-minute window, signature computed over `${timestamp}.${rawBody}` exactly as Architrak/Archidoc do on their inbound rules. Missing either header → receiver returns 401; we treat 401 as non-retryable (do not poison the queue).

- Body size cap: 1 MiB on outbound webhooks (signed PDF travels by URL, never inline).
- Idempotency: `eventId` UUID minted once at delivery creation, reused on every retry. Receivers dedup on `eventId`.
- Retry: 3 attempts, 1s/3s backoff, 10s per-attempt timeout, 5xx + network only. After 3 failures the delivery is marked `dead_letter` in `webhook_deliveries` with the last error captured; admin dashboard surfaces a "Retry" button.

This is the **single contract** both Archidoc (existing) and Architrak (new) consume. Archidoc: please confirm in Round 2 that adding the mandatory timestamp header is acceptable on your side too — your existing receiver already enforces it for inbound, but you'll need to start verifying it on the Archisign-sourced channel as well.

3.5 Signed PDF — 10-year retention commitment + stable re-mint endpoint
Direct answer to Archidoc's §6.1, which they flagged as "the single biggest unknown":

- **Yes, Archisign commits to a 10-year retention floor for signed PDFs.** Originals can age out (we can discuss a 90-day floor for unsigned originals separately); signed envelopes are retained for 10 years from `signedAt` per French decennial liability. We will add this as an explicit lifecycle rule with an annual integrity check job, not just a default Object Storage configuration.
- **New endpoint: `GET /api/v1/envelopes/:envelopeId/signed-pdf-url`**, `X-API-KEY` auth, returns `{ url, expiresAt }` where `url` is a freshly-minted 15-minute signed URL pointing at the envelope's signed PDF in Object Storage. Idempotent, no side effects. Both Architrak and Archidoc can call this at any point in the 10-year window.
- **Webhook payload includes a 15-minute `signedPdfFetchUrl`** at delivery time (matches Architrak's §5.2.signed shape). If retries land beyond the 15-minute window, the receiver re-mints via the endpoint above using `envelopeId`.
- We will **not** push the bytes inline in the webhook. Bytes flow by URL; the URL is re-mintable.

This means Archidoc's `archivedPdfStoragePath` column is a **nullable convenience**, not a mandatory ingest-time write. They can decide to mirror or not based on their own resilience posture.

3.6 Identity-verification trail in the `envelope.signed` payload
Direct answer to Archidoc's §6.2 ask. Today every signing already captures: OTP-issued-at, OTP-verified-at, signer IP, signer user-agent, `lastViewedAt`, `signedAt`, and the deterministic `authenticationId` from `SecurityService.generateAuthenticationId(signerId, envelopeId, signedAt)`. We will surface them on the `envelope.signed` webhook payload as a structured `identityVerification` block:

```
{

 "identityVerification": {

   "method": "email_otp",

   "otpIssuedAt": "ISO-8601",

   "otpVerifiedAt": "ISO-8601",
```

```
    "signerIpAddress": "203.0.113.42",

    "signerUserAgent": "Mozilla/5.0 ...",

    "lastViewedAt": "ISO-8601",

    "signedAt": "ISO-8601",

    "authenticationId": "deterministic-string"

  }

}
```

Architrak echoes this verbatim into their work-authorisation webhook to Archidoc per Architrak's §5.3. Archidoc lands it in `insurance_audit_log`. The decennial trail closes. Note: PII (IP, UA) is included because French regulatory expectation for e-signature evidence requires it; receivers must store accordingly.

3.7 Query loop — pause-and-resume, not re-create
Direct answer to Architrak's §6.4 — and important because it determines whether Architrak's `client_checks` table needs to track an `archisignEnvelopeId`.

- **Today**: when a signer raises a query, the envelope status flips to `queried`, the signer cannot continue signing until the architect responds, the architect responds via the admin UI, signer re-enters their OTP and resumes from where they left off. The envelope, signers, annotations, and access token all persist. No new envelope is created.
- **Therefore, Architrak's `client_checks` table SHOULD track `archisignEnvelopeId`.** A query is a pause, not a teardown. The same envelope resumes post-resolution.
- **We will add `envelope.query_resolved`** (or `envelope.resumed`, name TBD in Round 2) as a fifth event so Architrak can close their `client_checks` row reactively rather than polling.
- Architrak's preference that "the dialogue lives in Architrak, not Archisign" is fine in principle, but Archisign cannot give that up unilaterally — our admin UI already exposes the dialogue and our audit trail captures it. Proposal: both surfaces remain operative, and the `query_raised` webhook to Architrak carries the raw `queryText` so Architrak can mirror it into their `client_checks` row immediately. The architect can resolve in either UI; Archisign emits `query_resolved` either way. Open to challenge.

3.8 Decline reason capture
When `envelope.declined` fires, payload includes `declineReason: string` from a mandatory free-text field on the signer-side decline action (and the admin-side equivalent). Architrak's §4 maps this to their `voidReason`. Locked.

4. Endpoint contracts Archisign commits to
4.1 Inbound: Architrak → Archisign — create envelope (extends existing endpoint)
`POST /api/v1/envelopes/create`

Headers: `X-API-KEY: <ARCHITRAK_API_KEY>`

Body (additive over today's schema, no breaking changes for Archidoc):

```
{
 "subject": "Devis 2024-045 — Lot GO",
 "externalRef": "architrak:devis:1234",
 "signers": [{ "email": "jean.dupont@example.com", "fullName": "Jean Dupont" }],
 "pdfFetchUrl": "https://architrak.../signed-url-15min...",
 "pdfBase64": null,
 "webhookUrl": "https://architrak.../webhooks/archisign",
 "expiresInDays": 30,
 "metadata": {
   "architrakDevisId": 1234,
   "archidocProjectId": "abc-123",
   "archidocContractorId": "xyz-789"
 }
}
```

Response 201:

```
{
 "envelopeId": 5678,
 "status": "draft",
 "signers": [{ "id": 9012, "email": "...", "accessToken": "...", "signingUrl": "https://archisign.../sign/<token>" }],
 "createdAt": "ISO-8601"
}
```

Note: our `envelopeId` is an integer (`generatedAlwaysAsIdentity`), not a UUID. Architrak's §3.5 mention of "UUID" envelope IDs is inaccurate against current schema. We can migrate to UUID if the user/Architrak prefer it for cross-app aesthetics; **proposal: integer stays, Architrak treats as opaque `string | number`**. Open to challenge in Round 2.

4.2 Inbound: Architrak → Archisign — explicit envelope send
`POST /api/v1/envelopes/:envelopeId/send` — Architrak's signal to actually email the signer (created envelopes start in `draft` and require a separate send action; this matches our admin UI behaviour and avoids accidental sends if the create call partially completed). Idempotent.

4.3 Inbound: Architrak → Archisign — re-mint signed PDF URL
`GET /api/v1/envelopes/:envelopeId/signed-pdf-url` — see §3.5. Both Architrak and Archidoc may call.

4.4 Outbound: Archisign → Architrak — envelope status callback
`POST <webhookUrl-from-create>`

Headers (every event):

- `X-Archisign-Signature: sha256=<hex of`${timestamp}.${rawBody}`>`
- `X-Archisign-Timestamp: <unix ms>`

Body — common envelope:

```
{

 "eventId": "uuid-stable-across-retries",

 "envelopeId": 5678,

 "externalRef": "architrak:devis:1234",

 "metadata": { "architrakDevisId": 1234, "archidocProjectId":
"abc-123" },

 "event": "envelope.signed | envelope.declined | envelope.expired |
envelope.queried | envelope.query_resolved",

 "occurredAt": "ISO-8601"

}
```

Per-event additions:

- `envelope.signed`: `signedPdfFetchUrl` (15 min), `signedPdfFetchUrlExpiresAt`, `identityVerification` block (§3.6).
- `envelope.declined`: `declineReason`, `declinedBy: "signer" | "admin"`.
- `envelope.expired`: `expiredAt`.
- `envelope.queried`: `queryText`, `signerEmail`.

- `envelope.query_resolved`: `resolvedAt`, `resolvedBy`.

Receiver returns 200 within 5s on first delivery and on every retry (idempotent). 401 on bad signature or stale timestamp (non-retryable). 410 on unknown envelope (non-retryable, dead-letter on our side). All other 4xx non-retryable. 5xx and network errors retried per §3.4.

4.5 Outbound: Archisign → Archidoc — unchanged
Archidoc's existing inbound channel (`POST /api/webhooks/archisign`) keeps working with the **same** payload shape above. The only behaviour change for Archidoc is the new mandatory `X-Archisign-Timestamp` header (per §3.4). They already verify it on their own inbound from Architrak, so the verification code is reusable; the secret remains `ARCHISIGN_WEBHOOK_SECRET` on their side.

5. Archisign outcome mapping (acceptance of Architrak's §4 table)
Accepted, with two clarifications:

- `envelope.expired` → Architrak's `signOffStage → approved_for_signing` and architect resends. This requires Architrak to call `POST /api/v1/envelopes/:envelopeId/send` again rather than re-creating the envelope. We can support a "resend" via the existing endpoint without minting a new envelope; the access token is regenerated and a fresh OTP issued on next signer access. Open to challenge if Architrak prefers a fresh envelope on expiry — tradeoff is loss of the original audit chain.
- `envelope.queried` → Architrak clones into `client_checks` **and** records `archisignEnvelopeId` so the resume call (`query_resolved` event) can close the row. Per §3.7.

6. What Archisign needs from each counterparty in Round 2
From Architrak

1. **Acceptance of the additive `/api/v1/envelopes/create` contract in §4.1**, including: integer `envelopeId` (or formal request to migrate to UUID), the `metadata` blob convention for cross-app identifiers, and `pdfFetchUrl` semantics (server-side fetch, 60s timeout, 30 MB cap).
2. **Confirmation that Architrak signed-URLs survive ≥60s** — if your URL expiry is shorter than our fetch budget, we'll race.
3. **Acceptance of the timestamp-header upgrade in §3.4** on the inbound side (mandatory `X-Archisign-Timestamp`, signature over `${timestamp}.${body}`).
4. **Acceptance of the query-loop pause-and-resume model in §3.7** and confirmation that `client_checks` will store `archisignEnvelopeId`.
5. **Confirmation that email is acceptable for OTP delivery** (today's only channel) — your §6.7 asked about SMS; we are not adding SMS in this scope. If client-contact policy demands SMS, that is a Round 2+ feature with its own dependencies.
6. **Acceptance of the explicit two-step create+send flow in §4.2** rather than auto-send-on-create.

From Archidoc

1. **Acceptance of the timestamp-header upgrade in §3.4** on your existing Archisign inbound channel — same change as Architrak, applied to your already-live integration. We can ship behind a feature flag during the cutover if needed.
2. **Acceptance of the 10-year retention commitment + re-mint endpoint in §3.5** as sufficient for your decennial liability requirement, with `archivedPdfStoragePath` remaining a nullable convenience column on your side rather than a mandatory ingest-time copy.
3. **Confirmation that the `identityVerification` block in §3.6 is sufficient** for the trail you want echoed via Architrak — or specify additional fields you need.

4. **Confirmation of one-contract two-callers in §3.1** — i.e. you do not want a second, Archidoc-specific webhook variant when Architrak's channel evolves.

7. Archisign's open assumptions (please challenge in Round 2)

- **One envelope, one signer, in this workflow.** Both Architrak and Archidoc concur. Our schema supports multi-signer; we will not exercise it for the devis flow.
- **Email-only OTP.** No SMS, no TOTP, no FIDO. Email channel is Gmail via the existing connector.
- **`externalRef` is a free-form string** Architrak controls. Recommended convention `"architrak:devis:<id>"`, but we don't parse it.
- **`metadata` is a JSON blob** we round-trip on every webhook delivery without interpretation. Receivers MUST NOT trust its contents over what they themselves know — it's a convenience, not a source of truth.
- **No financial side effects on the Archisign side, ever.** We don't know, don't compute, and don't surface any money figure. The `envelope.signed` payload carries identity evidence and a PDF URL, nothing else of substantive value.
- **Decline is terminal.** A declined envelope cannot be "re-opened"; Architrak creates a new envelope (and a new `architrakDevisId` per their `client_rejected` semantics).
- **`expired` is a soft state.** The architect can resend the same envelope (per §5 clarification) or void it via Architrak. Archisign does not auto-void.
- **`webhookUrl` is per-envelope, supplied by the caller at create time**, not configured globally per tenant. Allows Architrak and Archidoc to evolve their webhook routes independently.

8. Archisign's confidence and reliability commitments

- **Service-layer enforcement is non-negotiable.** Per `ARCHISIGN_ARCHITECTURE.md`: PDF fetch logic ships in a new `services/PdfTransport.ts`; webhook verification middleware in `middleware/verifyArchitrakSignature.ts` (when we eventually accept inbound from Architrak — not in this scope but mentioned for completeness); expiry sweeper in `services/EnvelopeExpiry.ts`. No `pdf-lib`, no `fetch`-for-webhooks, and no crypto in route handlers. Routes stay under 1,000 lines (currently 992 — we will refactor before adding the new endpoints rather than blow the budget).
- **Outbound webhook signature + timestamp is non-negotiable** once Round 2 ratifies the contract. Both headers always present. Receivers that don't verify both will eventually be left behind — but that's a Round 3 enforcement decision, not a Round 1 stick.
- **10-year retention is a contractual commitment** backed by an annual integrity check, not a default GCS lifecycle setting. If we ever cannot honour it for an envelope (storage corruption, tenant deletion request), the affected envelopes are surfaced through the API with a structured error code, never silently dropped.
- **Idempotency is a stable-`eventId` discipline.** Same `eventId` across retries, persisted in `webhook_deliveries`, never re-minted. Dead-letter is visible in the admin UI; replay is one click.
- **No `any` in new cross-app code.** The new `shared/integrations/archisign.ts` (or equivalent path) will export named TypeScript interfaces for the create payload, the five webhook event payloads, and the re-mint URL response. Both Architrak and Archidoc consume.
- **Schema changes ship as committed Drizzle migrations.** New columns this scope: `envelopes.expiresAt`, `envelopes.declineReason`, `envelopes.origin`. New table: `webhook_deliveries`. Status enum gains an `expired` value. All gated behind a single migration file.
- **Test count grows.** New service files ship with tests covering: PDF fetch happy path + timeout, webhook signature+timestamp generation + verification round-trip, expiry

sweeper transitions, idempotent delivery on retry, dead-letter behaviour, identity-verification block construction.

Two flags before we go

1. **The integer `envelopeId` vs UUID question** in §4.1 is the single decision that touches Architrak's mirror schema. If they want UUIDs, we migrate now (additive — keep the integer PK, add a UUID column, expose UUID in the public API). If integer is fine, we save the migration. **Architrak: pick one in Round 2.**
2. **The mandatory timestamp-header upgrade in §3.4** is a contract change that affects Archidoc's already-live integration with us, not just the new Architrak channel. We can ship it behind a one-release feature flag (`x-archisign-signature-v2` header parallel to the v1 header) so Archidoc can cut over without coordinated downtime. **Archidoc: confirm the flag-and-cut-over plan or propose an alternative.**

That's the response — ready to paste into the shared document next to Architrak's and Archidoc's. No code has been written, no schema changes have been made, no tasks have been created; this is a planning artifact only. When you have the consolidated three-way set, share them back and I will produce Archisign's Round 2.

ROUND 2

ARCHITRAK — ROUND 2 RESPONSE

ARCHITRAK — Round 2 Response (revised): Devis Sign-Off Workflow Contract

1. Headline summary

With Archisign's full Round 1 in hand, the Round 2 picture is much closer to "we have a contract" than "we still have unknowns." Two of my own Round 1 positions are revised; everything else converges. Specifically:

1. **Insurance gate revision (still flagged for the user).** My Round 1 §3.1 said "mirror-only." I am revising to **mirror + live verdict + override + audit**, accepting Archidoc's §3.1 argument. The user originally locked "mirror-only," so this is the one Round-1 lock I'm asking to re-open. See §6 below.
2. **Retraction of my Round 1 §2 claim that the insurance mirror is "kept fresh by Archidoc's existing `contractor.updated` webhook."** Archidoc clarified that outbound webhook does not exist on their side. Architrak is in pull-only mode against `GET /api/sync/contractors`. Adopted.

Everything else in this Round 2 is acceptance of contract terms now visible in full.

2. Acceptances — locked

These are now contract terms unless Round 3 overturns them.

#	Topic	Architrak's position
A1	Live insurance verdict mandatory at the gate (Archidoc §3.1)	Accepted with override semantics — see §3

A2	`primaryClient: {name, email} | null` on Archidoc's project sync (Archidoc §3.2)	Accepted exactly as specified
A3	Work-authorisation payload additions (Archidoc §3.3)	Accept all four: `archidocDqeExportId`, `contractorSiret`, `devisReference`, `insuranceOverride`
A4	Consumer-side secret naming (Archidoc §3.4)	Accepted. Architrak's `ARCHIDOC_AUTHORIZATION_WEBHOOK_SECRET` ≡ Archidoc's `ARCHITRAK_WEBHOOK_SECRET`
A5	Identifiers as opaque strings (Archidoc §3.6)	Accepted. SIRET display-only
A6	Receiver-owned endpoint paths are authoritative	Architrak adopts `POST /api/integrations/architrak/work-authorisations` (Archidoc) and `POST /api/v1/envelopes/create` (Archisign)
A7	One Archisign contract, two callers, distinguished by `X-API-KEY` (Archisign §3.1)	Accepted. Architrak uses `ARCHITRAK_API_KEY`
A8	PDF transit = `pdfFetchUrl` (signed-URL pull) (Archisign §3.2)	Accepted. **Architrak commits to a 15-minute signed-URL TTL** — well above Archisign's 60s server-side fetch budget
A9	Webhook event taxonomy with five events (Archisign §3.3 + §3.7)	Accepted. Architrak handles `envelope.sent`, `envelope.queried` (alias `query_raised`), `envelope.query_resolved`, `envelope.declined`, `envelope.expired`, `envelope.signed`
A10	Webhook security with mandatory timestamp header, ±5 min window (Archisign §3.4)	Accepted. Symmetric on inbound and outbound

A11	eventId-based idempotency, both directions	Accepted. Architrak's `webhook_events` dedup table covers inbound; outbound mints a UUID per delivery, persisted in a new `webhook_deliveries_out` table for retries and dead-letter
A12	Email-only OTP (Archisign §6.5)	Accepted. No SMS scope here. My Round 1 §6.7 question is closed
A13	**10-year retention floor + `GET /api/v1/envelopes/:envelopeId/signed-pdf-url` re-mint endpoint** (Archisign §3.5)	**Accepted. Architrak does NOT mirror signed PDFs locally. We persist `archisignEnvelopeId` and re-mint on demand if a stored fetch URL has expired**
A14	**`identityVerification` block exact shape** (Archisign §3.6)	**Accepted verbatim. Architrak echoes the eight-field block into the `signerIdentityTrail` property of the work-authorisation webhook to Archidoc, no transformation**
A15	**Pause-and-resume on the query loop, dual-surface dialogue** (Archisign §3.7)	**Accepted with one constraint — see §4 below**
A16	**Decline reason capture as `declineReason: string`** (Archisign §3.8)	**Accepted. Architrak maps it to existing `voidReason` column**
A17	**Two-step create + send flow on Archisign** (Archisign §4.2)	**Accepted. Architrak issues `POST /api/v1/envelopes/:envelopeId/send` as a separate, idempotent call. This actually fits well — it lets us persist `archisignEnvelopeId` and update `signOffStage` to `sent_to_client` only after the send succeeds**
A18	**Resend on expiry, not fresh envelope** (Archisign §5)	**Accepted. On `envelope.expired`, Architrak transitions back to `approved_for_signing` and the architect can re-trigger send via the same envelope. Original audit chain preserved**

A19	**Integer envelopeId stays — Architrak treats as opaque `string | number`** (Archisign §4.1, §8 flag 1)	**Accepted. Architrak's `devis.archisignEnvelopeId` will be a `text` column populated with `String(envelopeId)`. No migration burden on Archisign's side. My Round 1 §3.5 mention of "UUID" was inaccurate; retracted**

3. Insurance gate — revised (replaces my Round 1 §3.1)
Adopting Archidoc's defence-in-depth model. At the `approved_for_signing → sent_to_client` transition:

1. Architrak calls `POST /api/integrations/architrak/contractors/:contractorId/insurance-verdict` with `{projectId, intendedWorkLot?}`, bearer-authed using `ARCHIDOC_SYNC_API_KEY`.
2. Outcomes:
   - `200 + canProceed: true` → green light, proceed automatically
   - `200 + canProceed: false` → red gate, override prompt with mandatory reason
   - `404` → red gate, "contractor not associated with project," **non-overridable** (data fix required)
   - `503` or timeout >5s → red gate, **overridable** with prominent warning that Archidoc was unreachable
3. The mirror is not removed — it still feeds list badges and dashboard hints. It just stops being the gate's source of truth.
4. Architrak's `insurance_overrides` table gains: `liveVerdictHttpStatus` (int), `liveVerdictResponse` (jsonb, full body or `null`).
5. The `insuranceOverride` block on the work-authorisation webhook is populated whenever `overridden: true`, carrying both `mirrorStatusAtOverride` and `liveVerdictCanProceed` so Archidoc has both perspectives in their audit log.

4. Query-loop dialogue — accept dual-surface, with one constraint
Archisign's §3.7 makes a fair point: the query dialogue already lives in their admin UI and audit trail; they cannot give that up unilaterally. Architrak accepts the dual-surface model:

- **`envelope.queried` carries `queryText`** → Architrak mirrors immediately into a new `client_checks` row with `archisignEnvelopeId`.
- **`envelope.query_resolved`** → Architrak closes the matching `client_checks` row reactively, no polling.
- **The architect can resolve in either UI.** Architrak does not block the Archisign admin UI.

One constraint Architrak asks Archisign to honour: when an architect resolves through Archisign's UI, the `query_resolved` payload must include the architect's `resolverEmail` (or `resolvedByActor: "archisign_admin_ui"`) so Architrak's `client_checks.resolvedByUserEmail` can be populated correctly. Otherwise we get `client_checks` rows that close themselves with no attributable actor.

Architrak also commits to: **never emitting `query_resolved` from our side via Archisign API**. Closure is an architect action in either Architrak's UI or Archisign's UI; it never happens automatically based on, say, a `client_checks` row being marked resolved by a non-architect.

5. Endpoint contracts — final shape Architrak commits to

5.1 Outbound: Architrak → Archisign — create envelope

```
POST {ARCHISIGN_BASE_URL}/api/v1/envelopes/create

Headers:

 X-API-KEY: <ARCHITRAK_API_KEY>

 Content-Type: application/json

Body (matches Archisign §4.1 verbatim):

 {

   "subject": "Devis 2024-045 — Lot GO",

   "externalRef": "architrak:devis:1234",

   "signers": [

     { "email": "jean.dupont@example.com", "fullName": "Jean Dupont"
}

   ],

   "pdfFetchUrl": "https://architrak.../signed-url-15min...",

   "pdfBase64": null,

   "webhookUrl": "https://architrak.../webhooks/archisign",

   "expiresInDays": 30,

   "metadata": {

     "architrakDevisId": 1234,

     "archidocProjectId": "abc-123",

     "archidocContractorId": "xyz-789",

     "archidocDqeExportId": "dqe-export-uuid",

     "devisReference": "Devis 2024-045"
```

```
    }

  }

Response 201: { envelopeId, status: "draft", signers[].accessToken,
signers[].signingUrl, createdAt }
```

Followed immediately (and independently retryable) by:

```
POST {ARCHISIGN_BASE_URL}/api/v1/envelopes/:envelopeId/send

Headers: X-API-KEY: <ARCHITRAK_API_KEY>
```

Architrak transitions `signOffStage → sent_to_client` only after the `/send` returns 2xx.

5.2 Inbound: Archisign → Architrak — envelope status callback
`POST /webhooks/archisign` on Architrak's side. HMAC-SHA256 over `${timestamp}.${rawBody}`, headers `X-Archisign-Signature: sha256=<hex>` and `X-Archisign-Timestamp: <unix ms>`. Architrak:

- Returns 200 within 5s on first delivery and on every retry, dedup'd by `eventId`
- Returns 401 on missing/invalid signature or stale timestamp (non-retryable)
- Returns 410 on unknown `envelopeId` (non-retryable)
- Maps the `event` field as follows:
    - `envelope.sent` → audit-only, no `signOffStage` change (we already set `sent_to_client` after our `/send` 2xx)
    - `envelope.queried` (or alias `query_raised`) → open new `client_checks` row, `signOffStage` stays `sent_to_client`
    - `envelope.query_resolved` → close matching `client_checks` row, `signOffStage` unchanged
    - `envelope.signed` → `signOffStage → client_signed_off`, persist the full `identityVerification` block, fire outbound work-authorisation webhook to Archidoc
    - `envelope.declined` → `signOffStage → void`, persist `declineReason` into `voidReason`
    - `envelope.expired` → `signOffStage → approved_for_signing`, surface a "resend" action in the UI

5.3 Outbound: Architrak → Archidoc — work-authorised notification

```
POST
{ARCHIDOC_BASE_URL}/api/integrations/architrak/work-authorisations

Headers:

 X-Architrak-Signature: sha256=<hex of ${timestamp}.${rawBody}, key =
ARCHIDOC_AUTHORIZATION_WEBHOOK_SECRET>

 X-Architrak-Timestamp: <unix ms>
```

```
Body:

 {

   "eventId": "uuid-stable-across-retries",

   "architrakDevisId": 1234,

   "archidocProjectId": "abc-123",

   "archidocContractorId": "xyz-789",

   "archidocDqeExportId": "dqe-export-uuid",     // null if not
captured

   "devisReference": "Devis 2024-045",

   "contractorSiret": "12345678901234",

   "signedAt": "2026-04-25T12:34:56.789Z",

   "signedPdfFetchUrl": "https://archisign.../signed-url-15min...",

   "archisignEnvelopeId": "5678",                // string-cast of
integer per A19

   "amountHt":  "12345.67",

   "amountTtc": "14814.80",

   "lotReferences": ["GO", "VRD"],

   "signerIdentityTrail": { ...verbatim Archisign
identityVerification block per A14... },

   "insuranceOverride": null | {

     "overridden": true,

     "overrideReason": "...",

     "overriddenByUserEmail": "architect@renosud.com",

     "mirrorStatusAtOverride": "expired",
```

```
      "liveVerdictHttpStatus": 200,

      "liveVerdictCanProceed": false

    }

  }
```

Architrak retries 3 times (1s/3s backoff, 10s per-attempt timeout, 5xx + network only), then dead-letters with a one-click admin "Retry" UI. Note: **`signedPdfFetchUrl` may have expired by the time Archidoc receives a retry** — if Archidoc needs the bytes after the URL TTL, they call Archisign's re-mint endpoint with `archisignEnvelopeId`. We carry the envelope ID specifically to make that path work.

6. Decision the user needs to confirm

The user's earlier instruction was "Insurance — mirror-only (with override + audit)." Architrak's §3 above **revises that** to "mirror + live verdict + override + audit," on Archidoc's argument. This is the only Round-1-locked decision being re-opened.

- **What changes for the user:** at "Send to client," the request takes ~30–80ms longer while we call Archidoc. If Archidoc is unreachable, the gate paints red and the architect must override explicitly with a reason.
- **What stays the same:** override is always available; audit trail is Architrak-owned; UX is unchanged in the green path.
- **Recommendation: accept the live verdict.** Defence in depth wins given decennial liability exposure.

If the user prefers to hold the line on mirror-only, Architrak will defend it in Round 3 — but Archidoc has called this non-negotiable from their side, so we'd be in a contract conflict that blocks the workflow.

7. DQE Export ID propagation — Architrak picks header-scrape

Per Archidoc's §6.4 choice: Architrak chooses **header-scrape** (`X-Archidoc-Dqe-Export-Id` from inbound Gmail) over the lookup endpoint. Zero-RTT at devis ingestion time, and Architrak's Gmail monitor already parses headers. We do not need or want the `match-dqe-export` lookup endpoint shipped. Field stays nullable end-to-end if the header isn't present.

8. Architrak's outstanding asks for Round 3

Almost everything is answered. Three small items remain:

1. **For Archisign — rate limits on `POST /api/v1/envelopes/create`.** If an architect approves a batch of 10–20 devis in one session, do we get throttled? If so, what's the burst budget? Architrak will queue if needed, but we'd rather know the limit than discover it.
2. **For Archisign — independent rotatability of `ARCHITRAK_API_KEY` vs `ARCHIDOC_API_KEY`.** Confirm that revoking one does not break the other.
3. **For Archidoc — confirmation on the §5.3 `insuranceOverride` extension** above (`liveVerdictHttpStatus` + `liveVerdictCanProceed` added). Does that fit your `insurance_audit_log` shape, or do you want it flattened?

Plus the answer to §8 flag 2 from Archisign — **Archidoc's preferred timestamp-header rollout plan** (the parallel `x-archisign-signature-v2` flag-and-cut-over Archisign proposed). This is purely between Archidoc and Archisign; Architrak doesn't have an existing channel to

migrate, so we're indifferent.

9. Architrak's Round 2 commitments (delta from Round 1)

- Schema migration package (one atomic Drizzle migration):
  - `client_checks`, `client_check_messages`, `client_check_tokens` tables
  - `insurance_overrides` table with the two new live-verdict columns
  - `devis.archidocDqeExportId` (text, nullable)
  - `devis.archisignEnvelopeId` (text, nullable, indexed) — string-cast of Archisign's integer
  - `projects.clientContactName`, `projects.clientContactEmail` (text, nullable, mirrored from Archidoc's `primaryClient`)
  - 3 new `signOffStage` enum values: `client_review_in_progress`, `client_agreed`, `client_rejected`
  - `webhook_deliveries_out` table for outbound idempotency + dead-letter
- New services in `server/services/`:
  - `insurance-gate.service.ts`
  - `archisign-client.service.ts` (handles create + separate /send + re-mint URL fetch)
  - `work-authorisation.service.ts`
  - `client-checks.service.ts`
- New middleware: `verify-archisign-signature.ts`, mirroring the existing pattern for the Archidoc inbound verifier
- New routers (thin): `routes/webhooks-archisign.ts`, `routes/client-checks.ts`, `routes/insurance-gate.ts`
- All 119 existing tests stay green. New work adds tests for: insurance-gate decision matrix, Archisign client retry behaviour, two-step create+send orchestration, work-authorisation idempotency, dead-letter UI, client-checks lifecycle, HMAC verifier happy/sad paths, eventId dedup, dual-surface query close attribution

10. Suggested Round 3 mechanics

We are very close to a frozen Inter-App Contract. I'd propose Round 3 produce a single **consolidated contract document** — not three more agent responses — covering:

- The locked endpoint contracts from §5 above and their counterparts on the other apps
- The full event taxonomy with payload shapes
- Schema deltas owned by each app
- Secrets and key-rotation policy
- A single delta log so anyone reading R1.x → R3 can see what changed and why

Once that exists, each agent can independently scope and propose its own implementation tasks against the frozen contract.

Three things blocking finalisation:

1. **User confirmation on the live-insurance-gate revision** (§6). Single most important decision point.
2. **Three small Round-3 asks** in §8 (Archisign rate limits, key rotation independence, Archidoc's audit-log shape acceptance).
3. **Archidoc's answer on the timestamp-header rollout plan** for their existing Archisign channel (Archisign §8 flag 2). Doesn't affect Architrak directly but is on the critical path for the contract to be signable by all three.

That's the revised Round 2. Two things to flag for you as the intermediary:

- **The retention question is fully closed.** Archisign committed to 10 years + a re-mint endpoint, which means neither Architrak nor Archidoc needs to copy the signed PDF locally. That's a meaningful simplification on both sides and removes Archidoc's "single biggest unknown."
- **The insurance-gate revision in §6 is still on you.** I'm recommending we accept Archidoc's defence-in-depth and overturn your earlier mirror-only lock. If you'd rather hold the line, tell me and I'll defend mirror-only in Round 3 — but Archidoc has flagged this as non-negotiable from their side, so you'd be choosing a contract conflict.

ARCHIDOC — Round 2 RESPONSE:

ARCHIDOC — Round 2 Response: Devis Sign-Off Workflow Contract

1. Headline summary

The contract is substantially converged. With Architrak's Round 2 and Archisign's Round 1 both in hand:

- **Archisign's 10-year retention commitment + stable re-mint endpoint** (Archisign §3.5) closes the single biggest unknown Archidoc flagged in Round 1. We go further than Archisign's offer of a "nullable convenience" archive column — we **drop the column entirely** and rely on Archisign's vault. Aligns with Architrak's parallel decision in their R2 §A13.
- **All four Round 1 payload additions accepted by Architrak** (`archidocDqeExportId`, `contractorSiret`, `devisReference`, `insuranceOverride`).
- **Live insurance verdict gate** acknowledged by Architrak as the path forward; user-decision now on the table. Archidoc remains non-negotiable on requiring it.
- **One technical correction to our R1 §3.4** — the HMAC scheme is `sha256(${timestamp}.${rawBody})`, not `sha256(rawBody)`. Aligning symmetrically with the Archisign / Architrak pattern.
- **One new payload field accepted** — `archisignEnvelopeId` (string-cast of Archisign's integer per Architrak A19) added to the work-authorisation contract and to our `work_authorisations` schema. Unlocks the re-mint path.
- **Archidoc commits to a concrete three-phase migration plan** for our existing `/api/webhooks/archisign` endpoint to Archisign's v2 signature scheme — see §4.

2. Acceptances — locked

#	Topic	Archidoc's position
D1	Architrak A1 — live insurance verdict at the gate	Confirmed as our hard requirement. User decision pending; see §3.

D2	Architrak A2 — derived `primaryClient` on `/api/sync/projects`	Locked per our R1 §3.2.
D3	Architrak A3 — four payload additions on work-authorisation webhook	Locked.
D4	Architrak A4 — consumer-side secret naming	Locked. Our env: `ARCHITRAK_WEBHOOK_SECRET`.
D5	Architrak A5 — opaque-string identifiers, SIRET display-only	Locked.
D6	Architrak A6 — receiver-owned endpoint paths	Confirmed. Ours: `POST /api/integrations/architrak/work-authorisations` and `POST /api/integrations/architrak/contractors/:contractorId/insurance-verdict`.
D7	Architrak A11 / Archisign §3.4 — `eventId` idempotency, both ways	Confirmed. New `webhook_events_in` table.
D8	Architrak A13 / Archisign §3.5 — 10-year retention + re-mint endpoint	Accepted. Drops `archivedPdfStoragePath` from our schema. See §3 for revised schema and §5.1 for direct answer to Archisign §6.2.
D9	Architrak A14 / Archisign §3.6 — `identityVerification` block	Accepted verbatim. See §5.2 for direct answer to Archisign §6.3.

D10	Architrak §7 — header-scrape for `archidocDqeExportId`	Accepted. Archidoc commits to setting Gmail header `X-Archidoc-Dqe-Export-Id: <dqe_extract_sends.id>` on every DQE extract email — see §6.4. The `match-dqe-export` lookup endpoint is **dropped**.
D11	Architrak A19 — `archisignEnvelopeId` as `string` (cast from integer)	Accepted. Added to our schema as `archisign_envelope_id varchar(64)` and to the §6.2 inbound payload contract.
D12	Archisign §3.1 — one Archisign contract, two callers via X-API-KEY	Locked. See §5.4 for direct answer to Archisign §6.4.
D13	Archisign §3.4 — v2 timestamp-header on outbound webhooks	Locked. See §4 for the rollout plan (direct answer to Archisign §6.1).

3. Insurance gate — user decision restated

This is a user decision, not Archidoc's, but Archidoc strongly supports Architrak's Round 2 recommendation to overturn the mirror-only lock and accept defence-in-depth. The case from Archidoc's perspective:

- **French decennial liability runs 10 years.** A signed contractor devis authorising work where Décennale was actually expired at the moment of signing is a concrete legal exposure for the architect. The polling mirror's staleness window — bounded by Architrak's poll interval, not by Archidoc's publish cadence — is exactly the gap where this exposure lives. The 30–80 ms latency of the live call is a rounding error against a 10-year liability tail.
- **Override is not removed.** If Archidoc is unreachable, the gate paints red and the architect overrides explicitly with a captured reason. Architrak's R2 §3 makes this explicit. The frictionless path the original brief required is preserved — what changes is that the architect's override is *informed* (they see Archidoc's last-known answer), not blind.
- **Defence-in-depth is what saved Archidoc in the 29 March 2026 production incident** (recorded in our project charter). Single-source-of-truth gates fail silently; redundant gates fail loud. We've lived this.

Recommendation to the user: accept Architrak's revision. We'd rather defend it once now than discover it via a contractor walking onto site without valid Décennale.

4. Archisign v2 signature migration — direct answer to Archisign §6.1

Acceptance: yes, with a concrete rollout plan. Archisign's §6.1 ask correctly identifies that our existing `/api/webhooks/archisign` channel needs the v2 timestamp upgrade. Verifying our current code: `server/middleware/verifyArchisignWebhook.ts` today does `sha256(JSON.stringify(req.body))` with no timestamp header — three real defects the v2 migration fixes naturally:

1. **Re-stringification drift** — current code re-stringifies parsed JSON; v2 raw-bytes scheme eliminates the drift.
2. **Silent no-op when secret missing** — current code calls `next()` if `ARCHISIGN_WEBHOOK_SECRET` is unset; v2 hard-requires the secret.
3. **403 vs 401 mismatch** — current code returns 403 on signature mismatch; v2 will align to 401 to match the cross-app convention.

Three-phase parallel-then-cut-over to fit within Round 3's contract-freeze window:

Phase	Action	Archisign side	Archidoc side	Duration
P1: Dual-emit	Both `X-Archisign-Signature` (current) and `X-Archisign-Signature-V2` + `X-Archisign-Timestamp` (new) on every delivery.	New: emit second header pair	No-op; existing verifier ignores unknown headers	2 weeks
P2: Dual-verify, prefer-v2	Verifier prefers v2 if both present (validates timestamp ±5 min), falls back to v1. Logs path on every request; metric counter `archisign_webhook_verify_path{version="v1"|"v2"}`. Run ≥1 week with `v1=0` count to confirm.	Continues dual-emit	Ship updated `verifyArchisignWebhook.ts`	1–2 weeks
P3: Cut-over	Archisign drops legacy header. Archidoc requires v2; rejects v1-only with 401.	Stop emitting legacy	Verifier hard-requires v2; legacy code path deleted	Permanent

Architrak's parallel inbound channel ships v2 from day one — no v1/v2 dual-verify on the new `verifyArchitrakWebhook.ts`. We absorb all migration cost on the Archisign legacy channel; the Architrak channel is born compliant.

Feature-flag offer accepted. Archisign offered "we can ship behind a feature flag during cutover if needed" — yes, we'd appreciate the dual-emit being controllable via a per-tenant

flag during P1 so we can isolate one Archidoc environment for verifier development without disturbing production traffic.

5. Direct answers to Archisign's other Round-2 asks

5.1 Archisign §6.2 — 10-year retention + re-mint endpoint

Accepted, and we go further. Archisign offered `archivedPdfStoragePath` as a "nullable convenience column" we could populate at our discretion. Archidoc's decision: **drop the column entirely.** We trust the 10-year retention commitment + the re-mint endpoint as a complete archival contract. Reasons:

- Symmetric with Architrak's R2 §A13 decision not to mirror locally — three apps, one vault, simpler audit story.
- Storing a copy creates a divergence risk: if Archisign's vault and our copy ever differ, which one is the legal record? Better to have one.
- Object Storage cost and webhook-handler runtime simplification.

The annual integrity check Archisign committed to in §3.5 is exactly the assurance we needed. If at any point Archisign cannot honour the 10-year SLA for a specific envelope (per their §8 "structured error code, never silently dropped"), we'll handle that as a remediation event — not a reason to mirror everything pre-emptively.

5.2 Archisign §6.3 — `identityVerification` block sufficiency

Accepted verbatim and sufficient. The 8 fields you proposed (`method`, `otpIssuedAt`, `otpVerifiedAt`, `signerIpAddress`, `signerUserAgent`, `lastViewedAt`, `signedAt`, `authenticationId`) cover the full decennial trail Archidoc needs. We will not request additional fields.

Archidoc lands the block on the new `work_authorisations.signer_identity_trail` JSONB column verbatim and surfaces it in the project audit log without flattening. PII handling per Archisign's note: stored within the project's existing access-controlled audit trail, not exposed externally.

5.3 Archisign §6.4 — one contract, two callers (no Archidoc-specific variant)

Confirmed. Archidoc does not want a second Archisign-specific webhook variant. We commit to:

- Continuing to use `ARCHIDOC_API_KEY` on outbound calls to Archisign as today (extended to call `GET /api/v1/envelopes/:envelopeId/signed-pdf-url` for re-mints).
- Receiving the same v2 webhook payload shape Architrak receives. The two callers are distinguished by API key on outbound and by `metadata` blob contents on inbound (we

read `metadata.architrakDevisId` only when present; absence means the envelope originated from our own e-sign module).
- Not requesting any tenant-discriminated payload fork.

Symmetric ask back to Archisign: please confirm in Round 3 that revoking `ARCHIDOC_API_KEY` does not affect `ARCHITRAK_API_KEY` operations and vice versa. This mirrors Architrak's §8.2 ask.

5.4 Architrak's Round-2 §8.3 — `insuranceOverride` shape extension

Yes, the extended shape fits Archidoc's `insurance_audit_log` verbatim and unflattened. When the work-authorisation webhook arrives with `insuranceOverride.overridden === true`, Archidoc writes:

```
await db.insert(insuranceAuditLog).values({
 contractorId,
 projectId,
 eventType: 'override_acknowledged',
 source: 'architrak',
 actorEmail: insuranceOverride.overriddenByUserEmail ?? null,
 metadata: insuranceOverride,  // verbatim, including liveVerdictHttpStatus + liveVerdictCanProceed
 createdAt: new Date(),
});
```

No flattening. The two new fields (`liveVerdictHttpStatus`, `liveVerdictCanProceed`) are the kind of forensic detail we want preserved verbatim for the architect's audit-log view.

6. Revised Archidoc schema (delta from R1 §4)

Two changes vs R1: drop `archivedPdfStoragePath` (per D8), add `archisignEnvelopeId` and `signerIdentityTrail` (per D11/D9).

```
// shared/schema.ts (additions — superseding R1 §4)
export const workAuthorisations = pgTable("work_authorisations", {
```

```
 id: varchar("id", { length: 255 }).primaryKey(),

 eventId: varchar("event_id", { length: 255 }).notNull().unique(),

 architrakDevisId: integer("architrak_devis_id").notNull(),

 archisignEnvelopeId: varchar("archisign_envelope_id", { length: 64
}).notNull(),  // NEW (D11)

 projectId: varchar("project_id", { length: 255 }).notNull()

   .references(() => projects.id, { onDelete: "restrict" }),

 contractorId: varchar("contractor_id", { length: 255 }).notNull()

   .references(() => contractors.id, { onDelete: "restrict" }),

 archidocDqeExportId: varchar("archidoc_dqe_export_id", { length: 255
}),

 devisReference: varchar("devis_reference", { length: 255 }),

 contractorSiret: varchar("contractor_siret", { length: 14 }),

 signedAt: timestamp("signed_at").notNull(),

 signedPdfFetchUrl: text("signed_pdf_fetch_url").notNull(),

 signedPdfFetchUrlExpiresAt:
timestamp("signed_pdf_fetch_url_expires_at"),

 amountHt: decimal("amount_ht", { precision: 14, scale: 2
}).notNull(),

 amountTtc: decimal("amount_ttc", { precision: 14, scale: 2
}).notNull(),

 lotReferences:
jsonb("lot_references").$type<string[]>().default([]),

 insuranceOverride: jsonb("insurance_override").$type<{

   overridden: boolean;
```

```
   overrideReason?: string;

   overriddenByUserEmail?: string;

   mirrorStatusAtOverride?: string;

   liveVerdictHttpStatus?: number;     // NEW per Architrak R2 §3 /
§8.3

   liveVerdictCanProceed?: boolean;    // NEW per Architrak R2 §3 /
§8.3

 } | null>(),

 signerIdentityTrail: jsonb("signer_identity_trail"),
// NEW (D9)

 createdAt: timestamp("created_at").defaultNow().notNull(),

}, (t) => [

 index("idx_work_auth_project").on(t.projectId),

 index("idx_work_auth_contractor").on(t.contractorId),

 index("idx_work_auth_architrak_devis").on(t.architrakDevisId),

 index("idx_work_auth_archisign_envelope").on(t.archisignEnvelopeId),
// NEW

]);

export const webhookEventsIn = pgTable("webhook_events_in", {

 eventId: varchar("event_id", { length: 255 }).primaryKey(),

 source: varchar("source", { length: 64 }).notNull(),  // "architrak"
| "archisign"

 receivedAt: timestamp("received_at").defaultNow().notNull(),

 bodyHash: varchar("body_hash", { length: 64 }).notNull(),
```

```
});
```

Both ship as a single Drizzle migration `drizzle/00XX_devis_signoff_workflow.sql`, gated by Rules 7–10 in our charter.

7. HMAC scheme correction — supersedes our R1 §3.4

Our Round 1 §3.4 said HMAC over "the raw request body, not parsed JSON." Both Architrak's R2 §5.2/§5.3 and Archisign's §3.4 use HMAC over **`${timestamp}.${rawBody}`** (timestamp included in signed material, more secure). Archidoc adopts the same scheme symmetrically:

- **Algorithm**: HMAC-SHA256, hex-encoded, computed over **`${timestamp}.${rawBody}`** where `timestamp` is the value of the `X-Architrak-Timestamp` header (decimal unix-ms string) and `rawBody` is the raw request bytes.
- **Header**: `X-Architrak-Signature: sha256=<hex>`.
- **Mandatory `X-Architrak-Timestamp: <unix ms>`** with **±5-minute replay window**, `timingSafeEqual` on recomputed signature. Missing either header → **401**.
- **Body size cap**: 1 MiB → **413**.
- **Idempotency**: dedup on `eventId` against `webhook_events_in`; second delivery returns **200** with `{deduplicated: true}`. Unknown `(architrakDevisId, archidocProjectId)` → **410**.
- **Implementation**: `server/middleware/verifyArchitrakWebhook.ts`, modelled after the post-migration `verifyArchisignWebhook.ts` v2. Both files share an extracted helper `server/middleware/hmacVerifier.ts` so we have one battle-tested implementation, not two.

8. Endpoint contracts — final shape Archidoc commits to (supersedes R1 §5)

8.1 Inbound: Architrak → Archidoc — live insurance verdict at the gate

```
POST
/api/integrations/architrak/contractors/:contractorId/insurance-verdi
ct

Auth: Authorization: Bearer <ARCHIDOC_SYNC_API_KEY>

Body: { "projectId": "abc-123", "intendedWorkLot": "GO" }
```

Response **200**:

```
{

 "contractorId": "xyz-789",

 "checkedAt": "2026-04-25T12:34:56.789Z",
```

```
 "insuranceStatus": "valid",

 "canProceed": true,

 "errors": [],

 "warnings": ["RC Pro expires in 47 days"],

 "decennale": { "endDate": "2027-12-31", "verifiedAt":
"2026-01-15T..." },

 "rcPro":     { "endDate": "2026-06-11", "verifiedAt":
"2026-01-15T..." }

}
```

- **404** if contractorId unknown or not associated with the supplied projectId. Architrak R2 §3 correctly classifies as non-overridable, data-fix required.
- **503** if our DB is degraded — Architrak R2 §3 correctly classifies as overridable with prominent warning.
- Reuses existing `validate-for-project` business logic verbatim.

8.2 Inbound: Architrak → Archidoc — work-authorised notification

```
POST /api/integrations/architrak/work-authorisations

Headers:

 X-Architrak-Signature: sha256=<hex of ${timestamp}.${rawBody}, key =
ARCHITRAK_WEBHOOK_SECRET>

 X-Architrak-Timestamp: <unix ms, ±5 min from server time>

Body:

{

 "eventId": "uuid-stable-across-retries",

 "architrakDevisId": 1234,

 "archidocProjectId": "abc-123",

 "archidocContractorId": "xyz-789",

 "archidocDqeExportId": "dqe-export-uuid",   // null if header-scrape
```

```
failed

 "devisReference": "Devis 2024-045",

 "contractorSiret": "12345678901234",

 "signedAt": "2026-04-25T12:34:56.789Z",

 "signedPdfFetchUrl": "https://archisign.../signed-url-15min...",

 "archisignEnvelopeId": "5678",              // string-cast of
integer (D11)

 "amountHt":  "12345.67",

 "amountTtc": "14814.80",

 "lotReferences": ["GO", "VRD"],

 "signerIdentityTrail": { /* Archisign identityVerification block,
verbatim (D9) */ },

 "insuranceOverride": null | {

   "overridden": true,

   "overrideReason": "string",

   "overriddenByUserEmail": "architect@renosud.com",

   "mirrorStatusAtOverride": "expired",

   "liveVerdictHttpStatus": 200,

   "liveVerdictCanProceed": false

 }

}
```

Returns **200** within 5 s on first delivery and on re-delivery (idempotent). **401** on bad signature, missing/stale timestamp. **410** on unknown `(architrakDevisId, archidocProjectId)`. **413** on body > 1 MiB. **503** during a degraded DB window — Architrak retries per their A11.

Side effects on **first** (non-deduplicated) delivery:

1. Insert into `work_authorisations`.
2. Append entry to `projects.auditLog` (kind: `'work_authorised'`).
3. If `insuranceOverride.overridden`, insert into `insurance_audit_log` per §5.4.
4. If `archidocDqeExportId` provided, mark `dqe_extract_sends.signed = true`.
5. Send Gmail notification to project's architect-of-record.

Zero state mutation on dedup'd retries.

8.3 Outbound: Archidoc → Archisign — re-mint signed PDF URL (NEW)

After the 15-min `signedPdfFetchUrl` expires, Archidoc re-mints on demand when the architect clicks "open signed devis" in our project audit view:

```
GET {ARCHISIGN_BASE_URL}/api/v1/envelopes/:envelopeId/signed-pdf-url

Header: X-API-KEY: <ARCHIDOC_API_KEY>
```

Response **200**: `{ "url": "...", "expiresAt": "ISO-8601" }`. We persist the new URL+expiry on `work_authorisations` (in-place update) so subsequent clicks within the same TTL window don't re-mint unnecessarily.

8.4 Outbound: Archidoc → Architrak — explicitly NONE

Restating R1 §5.3: no push channel from Archidoc to Architrak in this contract. Mirror-poll + live verdict + override webhook covers every case.

8.5 Outbound side: Archidoc emits `X-Archidoc-Dqe-Export-Id` Gmail header

Implementation commitment for D10 / Architrak R2 §7. Archidoc adds the header on every DQE extract email sent via `sendAndLabelEmail()` in `server/gmailService.ts`. Header value is the canonical `dqe_extract_sends.id`, echoed back as `archidocDqeExportId` in §8.2. If header-scrape fails (forwards, replies that strip headers, manual upload), the field is `null` end-to-end and the workflow does not block.

9. Archidoc's open items for Round 3

Three small items remain:

1. **Archisign Round 3 confirmation** of independent rotatability of `ARCHIDOC_API_KEY` vs `ARCHITRAK_API_KEY` (§5.3 above).
2. **Architrak Round 3 confirmation** of the v2 HMAC scheme correction in §7 — implicit acceptance assumed (your R2 §5.2 already uses this scheme), but flagging for the formal "yes."
3. **User Round 3 confirmation** of the live-insurance-gate revision (§3 above). Unblocks

the entire workflow.

10. Suggested Round 3 mechanics

Agreeing with Architrak's R2 §10: Round 3 produces a **single consolidated Inter-App Contract document**, not three more agent responses. Suggested ownership split:

- **Architrak** owns the lifecycle section (`signOffStage` enum, transition table, retry policy).
- **Archisign** owns the envelope-and-event taxonomy section (five event types, payload shapes, identity verification block, retention SLA, rate limits).
- **Archidoc** owns the schema-deltas section and the v2 migration plan (table additions across all three apps, env-var inventory, v2 signature rollout timeline per §4).
- **All three share** the secrets matrix — one row per secret, columns: name on each side, holder, rotation policy, last rotated.

Archidoc can produce its contributions to the consolidated doc within hours of the user's confirmation on the live-insurance-gate decision. Once the contract is frozen, each app independently scopes its implementation tasks.

Two flags before you go

- **The live-insurance-gate user decision is the single remaining blocker** for the workflow as a whole. Architrak now recommends acceptance; Archidoc is non-negotiable on requiring it. If the user holds the line on mirror-only, the contract conflicts.
- **The Archisign v2 signature migration in §4 is concrete and time-bound** — about 4–5 weeks end-to-end, can run in parallel with Architrak's implementation. Archidoc absorbs the migration cost on the legacy channel; Architrak's new channel ships v2 on day one. Worth Archisign and Architrak knowing in case they want to align timelines in the consolidated Round 3 doc.

That's the refreshed Round 2. No code or schema changes have been made — this remains a planning artifact only.

When the user confirms the live-gate decision and Round 3 freezes the contract, I'll come back with a proper task breakdown for Archidoc's implementation: the new `work_authorisations` and `webhook_events_in` tables, the two integration endpoints in §8.1 and §8.2, the v2 Archisign verifier migration in §4, the Gmail header emission in §8.5, and the re-mint endpoint client in §8.3 — five tasks, each with a clean acceptance criterion. You can then choose whether I work them directly or hand them to a task agent.

Optimized Agent memory
Worked for 3 minutes
Scroll to latest

ARCHISIGN — Round 2 RESPONSE:

ARCHISIGN — Round 2 Response: Devis Sign-Off Workflow Contract

1. Headline summary

The contract is essentially converged. With Architrak's R2 and Archidoc's R2 both in hand, every Round 1 position Archisign locked has been accepted by both counterparties:

- **All eight of my R1 design decisions** (one contract / two callers, `pdfFetchUrl` pull, six-event taxonomy with stable `eventId`, v2 HMAC with mandatory timestamp, 10-year retention + re-mint endpoint, identity-verification block, query-loop pause-and-resume, two-step create+send) — **locked by both Architrak (A1–A19) and Archidoc (D1–D13)**.
- **The integer-vs-UUID `envelopeId` flag** I raised in R1 §8 — Architrak A19 accepts integer-as-opaque-string. Closed.
- **The legacy-channel timestamp upgrade flag** I raised in R1 §8 — Archidoc §4 commits to a concrete three-phase parallel-then-cut-over plan and accepts my feature-flag offer. Closed.
- **The `archivedPdfStoragePath` "nullable convenience"** I offered Archidoc — they go further and drop it entirely, trusting Archisign's vault as the sole archive. Lighter contract, simpler audit story.

Three new commitments from Archisign in this round, plus direct answers to the three R3 asks I now have. Everything is in §3–§7.

2. Acceptances — locked

#	Topic	Archisign's position

S1	Architrak A1 — live insurance verdict at the gate	No-op for Archisign. We do not call Archidoc; we do not see the verdict. The `insuranceOverride` block round-trips through us only via the Architrak → Archidoc work-authorisation webhook (we never see it). Defer to user decision.
S2	Architrak §4 — dual-surface query loop with `resolvedBy` attribution	**Accepted with concrete payload commitment** — see §3.1.
S3	Architrak A8 — `pdfFetchUrl` 15-min TTL above our 60s fetch budget	Confirmed. We will never re-fetch beyond the initial create-time fetch; if the URL has expired by then, we 400 the create call.
S4	Architrak A9 — six-event taxonomy	Confirmed. **Round-3 canonical names locked in §3.2** — `query_raised` alias dropped.

S5	Architrak A10 + Archidoc §7 — HMAC over `${timestamp}.${rawBody}`, hex, `sha256=` prefix, ±5-min window	Locked. Identical scheme on every Archisign-emitted webhook.
S6	Architrak A11 — stable `eventId` UUID, persisted in our `webhook_deliveries` table, identical across retries	Locked.
S7	Architrak A13 / Archidoc D8 — neither counterparty mirrors signed PDFs locally; both rely on our 10-year retention + re-mint endpoint	**Accepted. Reinforced commitment in §3.3** — including the structured error code Archidoc references in their §5.1.
S8	Architrak A14 / Archidoc D9 — eight-field `identityVerification` block sufficient, no additional fields	Locked. Schema for the block is frozen as of this round.
S9	Architrak A17 — two-step `POST /create` + `POST /:envelopeId/send`, idempotent on the send side	Locked. We will return 200 on a re-send call against an already-`sent`/`viewed`/`queried` envelope rather than 409, so Architrak's retry path is safe.
S10	Architrak A18 — resend on `envelope.expired` reuses the same envelope, original audit chain preserved	Locked. We will rotate `accessToken` and re-issue OTP on next signer access; old

		token is invalidated.
S11	Architrak A19 — integer `envelopeId` stays, both sides treat as opaque string	Confirmed. Public API serialises `envelopeId` as an integer in JSON; receivers cast as needed.
S12	Archidoc §4 — three-phase v2 migration on the legacy `/api/webhooks/archisign` channel	**Accepted with implementation commitment in §4 below**, including the per-tenant feature flag for P1.
S13	Archidoc §5.3 — one contract two callers, no Archidoc-specific webhook variant	Locked. Only differentiator is the `X-API-KEY` value on inbound and the `metadata` blob contents on outbound.
S14	Archidoc §8.3 — `GET /api/v1/envelopes/:envelopeId/signed-pdf-url` with `X-API-KEY: ARCHIDOC_API_KEY`, returns `{ url, expiresAt }`	Locked. Same endpoint serves both callers; access control by `X-API-KEY` only — both keys can re-mint URLs for any envelope they can identify.

3. New Archisign commitments (delta from Round 1)

3.1 `envelope.query_resolved` payload — explicit attribution

Direct answer to Architrak's §4 constraint: when the architect resolves a query through

Archisign's admin UI, Archisign's `client_checks.resolvedByUserEmail` would be unattributable without an explicit field. We commit to the following payload shape on every `envelope.query_resolved` event:

```
{
 "eventId": "uuid-stable-across-retries",
 "envelopeId": 5678,
 "externalRef": "architrak:devis:1234",
 "metadata": { ... },
 "event": "envelope.query_resolved",
 "occurredAt": "ISO-8601",
 "resolvedAt": "ISO-8601",
 "resolverSource": "archisign_admin_ui" | "external",
 "resolverEmail": "architect@renosud.com" | null,
 "resolverActor": "architect" | "system",
 "resolutionNote": "string | null"
}
```

Notes:

- `resolverSource: "archisign_admin_ui"` whenever the action originates in our admin UI; `"external"` reserved for any future API-driven resolution path.
- `resolverEmail` is null only when `resolverActor === "system"` (e.g. an envelope-expiry sweeper closes a stale query in the future — not in this scope, but the field shape future-proofs it).
- `resolutionNote` carries the architect's free-text response to the signer if any. Optional.

Architrak: this gives you everything you need to populate `client_checks.resolvedByUserEmail`, `resolvedBySource`, and a free-text resolution audit

trail. Closed.

3.2 Webhook event taxonomy — Round-3 canonical names locked

Six events, canonical names frozen for the consolidated Round-3 contract:

Canonical name	Trigger	Architrak `signOffStage` mapping	Archidoc effect
`envelope.sent`	Successful `POST /:envelopeId/send`	audit-only (no transition)	audit-only
`envelope.queried`	Signer raises a query	open `client_checks` row	audit-only (visible in Archidoc's existing log)
`envelope.query_resolved`	Architect resolves in either UI	close matching `client_checks` row	audit-only
`envelope.declined`	Signer or admin declines with reason	`client_signed_off` → no; transition to `void` with `voidReason ← declineReason`	no work-authorisation fired
`envelope.expired`	Sweeper detects `expiresAt` passed	back to `approved_for_signing` for resend	no work-authorisation fired

`envelope.signed`	All required signers signed and PDF stamped	`client_signed_off` and fire Architrak → Archidoc work-authorisation	receives work-authorisation from Architrak (not directly from us)

The R1 `query_raised` alias is **dropped** in the canonical contract. Internal Archisign code may still log `query_raised` for backward compat, but the wire field is `event: "envelope.queried"` exclusively.

3.3 Retention SLA — structured error code commitment

Direct answer to Archidoc §5.1: they have dropped their archive column on the strength of our 10-year retention SLA, with the implicit reliance that we surface a structured error if we ever cannot honour it for a specific envelope. We commit:

- The annual integrity check job classifies any envelope it cannot prove retrievable as `retention_breach`.
- Both `GET /api/v1/envelopes/:envelopeId/signed-pdf-url` and `GET /api/v1/envelopes/:envelopeId` return **HTTP 410 Gone** with body:
- `{`
- `"error": "retention_breach",`
- `"envelopeId": 5678,`
- `"originalSignedAt": "ISO-8601",`
- `"detectedAt": "ISO-8601",`
- `"incidentRef": "string",`
- `"remediationContact": "string"`
- `}`
- We additionally **emit a one-shot `envelope.retention_breach` webhook** to the original `webhookUrl` recorded at create time, signed with the same v2 HMAC scheme. This is a seventh event type, but it is operationally exceptional — receivers may treat it as a remediation trigger, not part of the normal lifecycle. Archidoc and Architrak: please acknowledge this seventh event in your verifier whitelists for Round 3, even though you may never receive one.
- The error code is **not 404** specifically because 404 implies "we never had this" —

`retention_breach` says "we had it and we know what happened to it," which is the legally meaningful distinction.

3.4 Rate limits on `POST /api/v1/envelopes/create` — direct answer to Architrak §8.1

We commit to per-API-key rate limits, returning standard `429 Too Many Requests` with a `Retry-After: <seconds>` header on the response. Per-key budgets:

- **Sustained**: 60 requests / minute (1 RPS sustained).
- **Burst**: token bucket capacity of 30. An architect approving 10–20 devis in one session is well inside this budget.
- **Daily ceiling**: 5,000 requests / day per key. Soft alarm at 80 %, surfaced through the admin UI.
- The same budget applies independently to `POST /api/v1/envelopes/:envelopeId/send` and `GET /api/v1/envelopes/:envelopeId/signed-pdf-url` — bucket per (key, endpoint family) pair.

Implementation discipline: rate-limit middleware in `server/middleware/rateLimit.ts`, never inline in routes. Counters in-memory per-process for now (single-process Replit deployment); we will migrate to a shared store if and when we go multi-process.

If 60 RPM ever proves too tight for an architect's batch workflow, we'll raise the ceiling — flagging an unrealistic limit now would just defer the problem. Architrak: if you have a known peak you want me to plan against, name it in Round 3 and we'll set the budget there.

3.5 Independent key rotation — direct answer to Architrak §8.2 and Archidoc §5.3 / §9.1

Locked, both directions:

- The auth middleware accepts **any value present in the union** of `ARCHIDOC_API_KEY` and `ARCHITRAK_API_KEY` env vars. Each env var supports a comma-separated list to allow rotation-in-place (old + new key valid simultaneously during a rotation window).
- Revoking either env var (unsetting it, or removing a value from the list) **immediately invalidates only the keys in that var**; the other tenant continues unaffected. No deploy is required to rotate; the middleware reads the env on every request.
- Each request is tagged with the resolved tenant (`origin: "archidoc" | "architrak"`) and persisted on `envelopes.origin` (per R1 §3.1). Audit trail in

`audit_events` carries the same tag.

- Rotation discipline is **caller-owned**: Archisign does not coordinate rotations. If Architrak rotates `ARCHITRAK_API_KEY`, Archidoc is not informed (and does not need to be).

Archidoc's symmetric ask in §5.3 is identical — same answer covers it. Closed.

4. Legacy channel v2 migration — accepting Archidoc's three-phase plan

Direct answer to Archidoc §4. Their plan is sound and we adopt it verbatim:

Phase	Archisign side	Duration
P1: Dual-emit	Both `X-Archisign-Signature` (legacy v1, body-only) and `X-Archisign-Signature-V2` + `X-Archisign-Timestamp` (v2, `${timestamp}.${rawBody}`) on every delivery to legacy receivers. **New `ARCHISIGN_WEBHOOK_V2_TENANTS` env var** (comma-separated webhook URL prefixes or tenant tags) controls dual-emit per-tenant — Archidoc can opt one environment in for verifier development without touching production.	2 weeks
P2: Dual-verify on Archidoc side	We continue dual-emit unchanged. We expose a metric counter `archisign_webhook_emit_path{version="v1"|"v2"}` (we already log this).	1–2 weeks
P3: Cut-over	Drop the legacy header entirely. v2 headers only, on every delivery. Receivers that haven't migrated will fail signature verification → 401 → dead-letter on our side.	Permanent

Notes:

- **Architrak's channel is born v2-only.** No dual-emit, no migration phase. The new `verify-archisign-signature.ts` middleware on Architrak's side ships v2 from day one. Architrak A10 already commits to this.
- We emit the **same `eventId` and same body** under both v1 and v2 headers in P1 — they are two signature envelopes around one logical delivery. Archidoc's idempotency table should still dedup correctly.
- Webhook delivery code lives in `services/NotificationService.dispatchWebhook` per `ARCHISIGN_ARCHITECTURE.md`. The v1/v2 dual-emit is configured via the env var and a small `services/WebhookSignature.ts` helper; route handlers do not learn about either scheme. This keeps our charter intact.

If Archidoc wants a tighter timeline (say, one-week phases), we can compress to ~2.5 weeks total — flag in Round 3.

5. Outstanding asks from Archisign for Round 3

Almost nothing. Two small items, both housekeeping:

1. **From Architrak and Archidoc** — please both acknowledge the new seventh event `envelope.retention_breach` in §3.3. This is operationally exceptional but contractually present. If either of you objects, name an alternative remediation channel.
2. **From Architrak** — confirm the resend-on-expiry semantics in S10 above (rotated `accessToken`, fresh OTP on next access, old token invalidated). Architrak A18 implicitly accepts this; flagging for the formal "yes" so I can document it in the consolidated contract.

6. Archisign's Round-3 ownership claim

Agreeing with Architrak §10 and Archidoc §10: Round 3 produces a single consolidated Inter-App Contract document, not three more agent responses. Per Archidoc §10's suggested split, **Archisign owns**:

- The envelope-and-event taxonomy (the §3.2 table above, with full payload shapes per event including the new `envelope.retention_breach`).
- The `identityVerification` block schema (eight fields, frozen).
- The 10-year retention SLA + the structured `retention_breach` error code.
- The rate-limit policy in §3.4.

- The two-step create-then-send orchestration (request/response shapes for `POST /api/v1/envelopes/create`, `POST /api/v1/envelopes/:envelopeId/send`, `GET /api/v1/envelopes/:envelopeId/signed-pdf-url`).
- The v2 HMAC signature scheme spec (one helper file, identical math on all three apps).

Architrak owns the lifecycle/`signOffStage` section. Archidoc owns the schema-deltas section and the v2 migration timeline. All three of us co-own the secrets matrix (one row per secret, columns: name on each side, holder, rotation policy).

7. Two flags before we go

1. **The `envelope.retention_breach` seventh event in §3.3** is a small contract addition. It costs both counterparties one entry in their event-name whitelist; in exchange, Archisign converts an "unspeakable failure mode" into a structured remediation trigger that satisfies Archidoc's decennial-liability assurance posture. Worth confirming explicitly.
2. **The legacy-channel migration in §4 will take 4–5 weeks end-to-end** (matching Archidoc's estimate) and runs in parallel with Architrak's day-one v2 implementation. Archisign absorbs the dual-emit cost; Archidoc absorbs the dual-verify cost; Architrak absorbs neither. Worth aligning timelines in the consolidated Round 3 doc so all three apps know when Archisign's dispatch code briefly carries two header sets.

That's the response — ready to paste into the shared document next to Architrak's and Archidoc's R2. No code or schema changes have been made. Once Round 3 freezes the consolidated contract — i.e. the user confirms the live-insurance-gate revision and both counterparties acknowledge the seventh-event addition in §3.3 — I will come back with a concrete implementation task breakdown for Archisign covering: the `ARCHITRAK_API_KEY` middleware extension, the `pdfFetchUrl` server-side fetch service, the four new wire events (`envelope.declined`, `envelope.expired`, `envelope.query_resolved`, `envelope.retention_breach`) plus the `envelope.queried`-payload upgrade, the `webhook_deliveries` idempotency table + dead-letter UI, the v2 signature scheme + dual-emit migration, the `expiresAt` sweeper, the re-mint endpoint, the rate-limit middleware, and the annual integrity check job. You'll then choose whether I work them directly or hand them to a task agent.

Worked for 3 minutes

Scroll to latest

Ask questions, plan your work…

ROUND 3

ARCHITRAK — Round 3 Response: Devis Sign-Off Workflow Contract

1. Headline summary

The contract is frozen-ready from Architrak's side. Round 2 produced three new items that need an Architrak position:

1. **Archisign's seventh event** `envelope.retention_breach` (Archisign R2 §3.3) — accepted, with a defined Architrak handling path that does **not** mutate `signOffStage` (a signed devis remains signed) but does surface a remediation prompt and write to a dedicated audit table. See §3.
2. **Formal "yes" on resend-on-expiry semantics** (Archisign R2 §S10, asked in Archisign R2 §5.2) — confirmed. See §4.
3. **Formal "yes" on Archidoc's HMAC scheme correction** (Archidoc R2 §7, asked in Archidoc R2 §9.2) — confirmed. Architrak has already adopted `sha256(${timestamp}.${rawBody})` symmetrically across every inbound and outbound channel in this workflow, since R1. See §5.

Plus the answer to Archisign R2 §3.4's "name your peak" question for rate-limit planning. See §6.

Everything else from Round 2 is locked. The only remaining blocker for the workflow as a whole is the **user's confirmation on the live-insurance-gate revision** (Architrak R2 §6, supported by Archidoc R2 §3, supported by Architrak R3 below).

2. Acceptances — locked in this round

#	Topic	Architrak's position

AT1	**Archisign §3.2 — six-event canonical taxonomy** with `envelope.queried` (no `query_raised` alias on the wire)	Locked. Architrak's verifier whitelist accepts only the six canonical names plus the seventh below
AT2	**Archisign §3.1 — `envelope.query_resolved` payload schema** (`resolverSource`, `resolverEmail`, `resolverActor`, `resolutionNote`)	Locked. Architrak's `client_checks` schema gains `resolved_by_source`, `resolved_by_user_email`, `resolved_by_actor`, `resolution_note` columns to land the block 1-for-1 without flattening
AT3	**Archisign §3.4 — rate-limit policy** (60 RPM sustained, burst 30, 5,000/day, 429 + Retry-After)	Locked. Architrak's Archisign client honours `Retry-After`, queues with backoff, surfaces a "queue depth" hint in the admin UI for diagnostic visibility
AT4	**Archisign §3.5 — independent key rotation** with comma-separated env-var list	Locked. No coordination required; Architrak rotates `ARCHITRAK_API_KEY` on its own cadence
AT5	**Archisign §S9 — two-step create+send is idempotent on `/send`**, returns 200 on re-send against `sent`/`viewed`/`queried`	Locked. Architrak's outbound retry logic for the `/send` step is now safe by contract

AT6	**Archidoc §D8 — both apps drop local archive of signed PDFs**, rely on Archisign's vault + re-mint endpoint	Already in Architrak's R2 §A13; reaffirmed
AT7	**Archidoc §7 — HMAC scheme correction** (`sha256(${timestamp}.${rawBody})`, `sha256=` prefix, mandatory `X-Architrak-Timestamp`, ±5 min)	Confirmed verbatim. See §5 below
AT8	**Archidoc §8.5 — `X-Archidoc-Dqe-Export-Id` Gmail header** on every DQE extract email	Locked. Architrak's Gmail watcher reads the header (case-insensitive) at devis ingestion and persists to `devis.archidocDqeExportId`. Field stays nullable; absence does not block the workflow
AT9	**Archidoc §10 — Round 3 ownership split** (Architrak owns lifecycle, Archisign owns event taxonomy, Archidoc owns schema deltas + v2 migration, all three co-own secrets matrix)	Accepted. Architrak's contributions are in §7 below
AT10	**Archisign §3.3 — seventh event `envelope.retention_breach`**	Accepted with Architrak's specific handling — see §3
AT11	**Archisign §S10 / §5.2 — resend-on-expiry rotates access token, re-issues OTP, invalidates old token**	Confirmed — see §4

3. `envelope.retention_breach` — how Architrak handles it

Accepted as the seventh event in Architrak's verifier whitelist. The handling path is deliberately conservative because this is a vault-side notice about a previously-good signing event:

- **`signOffStage` is NOT mutated.** A devis at `client_signed_off` stays at `client_signed_off`. The signature was valid at the time it was applied; the breach is about future retrievability of the bytes, not about the legal validity of the past act. Any other behaviour would create an integrity problem in our own audit log.
- **A new dedicated table `signed_pdf_retention_breaches`** captures: `devisId`, `archisignEnvelopeId`, `originalSignedAt`, `detectedAt`, `incidentRef`, `remediationContact`, `receivedAt`. One row per breach event, indexed on `devisId` and `archisignEnvelopeId`.
- **Admin UI surface:** the affected devis gains a persistent red banner ("Signed PDF no longer retrievable from Archisign vault — incident `<incidentRef>`. Contact `<remediationContact>`.") on its detail page, plus a row in a new Settings → Integrations → "Vault Incidents" page listing all breaches. The architect can mark the incident "acknowledged" but cannot dismiss it; resolution requires confirmation from Archisign that the bytes have been recovered (out-of-band).
- **Outbound side-effect:** Architrak fires a downstream notification webhook to Archidoc's `/api/integrations/architrak/work-authorisations` channel only if the corresponding work-authorisation was previously delivered, with a new `event` discriminator field and `incidentRef`. Reasoning: Archidoc has dropped its archive column on the strength of the same retention SLA, and they need to know symmetrically. **Open question for Archidoc in §8 below** — confirm you want this re-notification rather than fetching it lazily from Archisign at signed-PDF-fetch time.
- **No automatic remediation.** Architrak does not attempt to re-fetch, re-mint, or re-create the envelope. The 410 with `retention_breach` body from Archisign is treated as a definitive statement; the architect remediates via the `incidentRef` channel.
- **Idempotency:** dedup'd on `eventId` like every other webhook. A re-delivery of the same breach event is a 200 no-op.

4. Resend-on-expiry — formal yes

Confirmed (Archisign §S10):

- On `envelope.expired`, Architrak transitions `signOffStage →`

`approved_for_signing` and surfaces a "Resend signing request" action in the UI.

- The architect clicks "Resend." Architrak issues `POST /api/v1/envelopes/:envelopeId/send` against the **same** envelope (no new create). Architrak does not touch the existing envelope ID or audit chain.
- Architrak understands and accepts: Archisign rotates the signer's `accessToken`, invalidates the previous token, issues a fresh OTP on next signer access. Architrak's stored `signers[].accessToken` from the original create response is therefore stale after a resend; Architrak does not surface or rely on it post-resend.
- Architrak will optionally also offer the architect a "Set new expiry" option which, if used, calls a small Architrak-side helper that issues a fresh `/create` (new envelope ID, new audit chain) — but **only as an explicit architect choice**, not automatically. Reasoning: 99% of expiries should reuse; the 1% case where the original signer is no longer the right person is the exception.

5. HMAC scheme — formal yes

Confirmed (Archidoc §7 / §9.2):

- All Architrak inbound and outbound HMACs in this workflow use `HMAC-SHA256` over `${timestamp}.${rawBody}`, hex-encoded, header value prefixed with `sha256=`.
- The signed material is the raw request bytes (not re-stringified parsed JSON) plus the literal `X-*-Timestamp` header value (decimal unix-ms string) joined with a literal `.`.
- Mandatory `X-{Architrak,Archisign}-Timestamp: <unix ms>` with ±5-minute replay window enforced via `crypto.timingSafeEqual` on the recomputed signature. Missing either header → 401 (non-retryable from the sender's perspective).
- Architrak extracts the verifier into `server/middleware/hmac-verifier.ts` (single helper) so `verify-archisign-signature.ts` and any future inbound verifier share one battle-tested implementation. This mirrors Archidoc's §7 plan to extract a single `hmacVerifier.ts`.
- Architrak's existing inbound channel from Archidoc already runs this scheme in production; no migration needed on Architrak's side.

6. Architrak's peak — for Archisign's rate-limit planning

Direct answer to Archisign §3.4 ("if you have a known peak you want me to plan against, name it in Round 3"):

- **Realistic batch peak: 10–25 devis approved in one architect session.** The

architect of record at our largest tenant typically processes a week's accumulated devis on Monday mornings; we have observed bursts up to ~25 in a 30-minute window. This is well inside Archisign's burst-30 token bucket and 60-RPM sustained budget.

- **Daily ceiling realism: <200 devis/day per tenant** even on the busiest days. Architrak does not anticipate hitting Archisign's 5,000/day ceiling under normal load. If a tenant ever does, that's a strong signal of misuse and we'd want to know.
- **No special accommodation requested.** The published budget is comfortable. If we hit 429s in production we'll surface them in Architrak's "Vault Incidents" admin view and coordinate with Archisign before asking for a raise.

7. Architrak's Round-3 lifecycle section (per Archidoc §10 ownership split)

For inclusion in the consolidated Inter-App Contract.

7.1 `signOffStage` enum — frozen

```
export const signOffStageEnum = pgEnum("sign_off_stage", [
 "received",                        // pre-existing
 "checked_internal",                // pre-existing
 "client_review_in_progress",       // NEW
 "client_agreed",                   // NEW
 "client_rejected",                 // NEW (terminal)
 "approved_for_signing",            // pre-existing
 "sent_to_client",                  // pre-existing — semantics
tightened
 "client_signed_off",               // pre-existing — semantics
tightened
 "void",                            // pre-existing (terminal)
]);
```

7.2 Transition table — frozen

From	To	Trigger	Side-effects
`received`	`checked_internal`	Architect confirms AI-extracted data	none
`checked_internal`	`client_review_in_progress`	Architect kicks off optional client review	open new `client_checks` row, send token-auth client portal email
`client_review_in_progress`	`client_agreed`	Architect marks resolved (no open `client_checks`)	close all `client_checks` rows
`client_review_in_progress`	`client_rejected`	Architect rejects (terminal)	close all `client_checks`, send rejection email
`checked_internal` | `client_agreed`	`approved_for_signing`	Architect approves for signing	none
`approved_for_signing`	`sent_to_client`	Architect clicks "Send to client" → **insurance gate** → Archisign `/create` → Archisign `/send`	persist `archisignEnvelopeId`, persist `signerIdentityTrail` placeholder
`sent_to_client`	`sent_to_client`	`envelope.queri`	open `client_checks` row mirroring

		`ed` from Archisign	`queryText`
`sent_to_client`	`sent_to_client`	`envelope.query_resolved` from Archisign	close matching `client_checks` row, populate attribution from `resolverSource`/`resolverEmail`/`resolverActor`/`resolutionNote`
`sent_to_client`	`client_signed_off`	`envelope.signed` from Archisign	persist `signerIdentityTrail`, fire outbound work-authorisation webhook to Archidoc
`sent_to_client`	`void`	`envelope.declined` from Archisign	persist `voidReason ← declineReason`
`sent_to_client`	`approved_for_signing`	`envelope.expired` from Archisign	surface "Resend" action; on resend, transition back to `sent_to_client` via existing envelope's `/send`
`client_signed_off`	`client_signed_off`	`envelope.retention_breach` from Archisign	open `signed_pdf_retention_breaches` row, surface red banner, fire downstream notification to Archidoc (per §3

			above)
`*` (non-terminal)	`void`	Architect manual void	persist `voidReason`

Terminal stages: `client_rejected`, `void`, `client_signed_off` (the latter is logically terminal but technically still receives one possible event — the retention-breach notice — without changing stage).

7.3 Insurance gate semantics — at `approved_for_signing → sent_to_client`

Per Architrak R2 §3, with the user's pending confirmation:

1. Architrak calls `POST /api/integrations/architrak/contractors/:contractorId/insurance-verdict` with `{projectId, intendedWorkLot?}`, bearer-authed via `ARCHIDOC_SYNC_API_KEY`.
2. Outcomes:
   - `200 + canProceed: true` → green, proceed
   - `200 + canProceed: false` → red, **overridable** with mandatory reason
   - `404` → red, **non-overridable** (data fix required: assign contractor to project)
   - `503` or timeout >5s → red, **overridable** with prominent "Archidoc unreachable" warning
3. Override writes to `insurance_overrides` table (Architrak-side) with `liveVerdictHttpStatus`, `liveVerdictResponse` (jsonb), `mirrorStatusAtOverride`, `mirrorSyncedAtAtOverride`, `overrideReason`, `overriddenByUserEmail`. The full block is echoed verbatim into the work-authorisation webhook's `insuranceOverride` field per Architrak R2 §5.3.
4. The mirror persists for list badges and dashboard hints; it is no longer the gate's source of truth.

7.4 Outbound retry policy (Architrak → Archisign and Architrak → Archidoc)

3 attempts, 1s/3s backoff, 10s per-attempt timeout, retry on 5xx + network errors only. 4xx fails immediately and dead-letters into Architrak's `webhook_deliveries_out` table. Admin UI exposes a one-click "Retry" button per dead-letter row. Stable `eventId` UUID across retries.

7.5 Inbound retry expectation (Architrak as receiver of Archisign callbacks)

Architrak returns 200 within 5s on first delivery and on every retry, dedup'd by `eventId` against `webhook_events` table. 401 on signature/timestamp issues (non-retryable). 410 on unknown envelope (non-retryable). All other errors map to 5xx so Archisign's own retry logic kicks in.

7.6 Architrak schema deltas — for the consolidated doc

(Same as Architrak R2 §9, with the §3 additions for retention-breach handling)

- New tables: `client_checks`, `client_check_messages`, `client_check_tokens`, `insurance_overrides`, `signed_pdf_retention_breaches`, `webhook_deliveries_out`
- New columns on `devis`: `archidocDqeExportId` (text), `archisignEnvelopeId` (text, indexed), `signerIdentityTrail` (jsonb)
- New columns on `projects`: `clientContactName`, `clientContactEmail` (both text, nullable, mirrored from Archidoc's `primaryClient`)
- Three new `signOffStage` enum values: `client_review_in_progress`, `client_agreed`, `client_rejected`
- Verifier whitelist for inbound Archisign events: the six canonical names from Archisign §3.2 plus `envelope.retention_breach`
- All shipped as one atomic Drizzle migration

8. Architrak's outstanding asks for the consolidated Round-3 doc

Two items only:

1. **For Archidoc — confirm the §3 retention-breach re-notification.** When Architrak receives `envelope.retention_breach` for a devis whose work-authorisation was previously delivered to Archidoc, Architrak proposes to fire a downstream notification to your `/api/integrations/architrak/work-authorisations` endpoint with a discriminator field (e.g. `event: "signed_pdf_retention_breach"`, plus `incidentRef`, `archisignEnvelopeId`, `architrakDevisId`). The alternative is for Archidoc to discover the breach lazily on next signed-PDF re-mint attempt. Architrak prefers proactive notification because the architect should know without having to click into the document. Confirm in Round 3.
2. **Cross-app secrets matrix — single source for the consolidated doc.** Archidoc §10 proposed all three of us co-own this. Suggested column set: secret name on each side, holder app, environments where it lives, rotation policy (in-place via

comma-separated list per Archisign §3.5), last rotated. Architrak can produce a first draft if helpful — flag it.

9. The one remaining workflow-level blocker

User confirmation of the live-insurance-gate revision. Both Architrak (R2 §6) and Archidoc (R2 §3) now formally recommend overturning the original "mirror-only" lock and adopting defence-in-depth. Archidoc has called the live verdict non-negotiable from their side. Without this confirmation:

- The contract cannot be frozen at the consolidated Round-3 stage.
- Architrak's `insurance_overrides` table cannot be finalised (the two `liveVerdict*` columns only exist if the live verdict exists).
- Archidoc cannot expose the `insurance-verdict` endpoint.

Once the user confirms, the consolidated Round-3 contract can be assembled within the day, and each agent can independently scope its implementation tasks against the frozen contract.

That's Round 3 from Architrak. Three flags for you as the intermediary:

- **The `envelope.retention_breach` handling in §3 introduces a new downstream notification from Architrak to Archidoc** that wasn't in any prior round. Worth surfacing to Archidoc explicitly in the next compile so they can confirm or push back in their Round 3.
- **The user-decision on the live insurance gate is now the single workflow-level blocker.** Both Architrak and Archidoc have lined up behind defence-in-depth. If you'd rather hold the line on mirror-only, it needs to come back to you for explicit reconfirmation — but expect Archidoc to escalate, since they have flagged it non-negotiable.
- **After you compile Round 3 with Archidoc's and Archisign's responses, the next natural step is the consolidated Inter-App Contract document** (one document, three-section ownership per Archidoc §10) rather than a Round 4 of agent dialogue. From that frozen contract Architrak can break down implementation tasks for you to either run directly or hand to a task agent.

Worked for 3 minutes

Scroll to latest

ARCHIDOC — Round 3 RESPONSE:

ARCHIDOC — Round 3 Response: Devis Sign-Off Workflow Contract

1. Headline summary

The contract is frozen-ready from Archidoc's side. With Architrak's R3 and Archisign's R2 both in hand, every Round 2 position Archidoc locked is accepted by both counterparties, and the only workflow-level blocker is the user's confirmation on the live-insurance-gate revision (Architrak R2 §6, Archidoc R2 §3, Architrak R3 §9 — all three have lined up behind defence-in-depth).

Three new items needed an Archidoc position in this round:

1. **Archisign's seventh event** `envelope.retention_breach` (Archisign R2 §3.3) — accepted on both paths Archidoc receives webhooks: the existing direct-from-Archisign channel (for plans/contracts envelopes Archidoc itself created) and the new via-Architrak downstream path (for devis envelopes Architrak created). See §3.
2. **Architrak's proposal of proactive re-notification to Archidoc on retention-breach** (Architrak R3 §3 + §8.1) — accepted, with a defined discriminator schema and the dedup story spelled out so the two paths cannot double-fire. See §4.
3. **Archisign's confirmation of independent key rotation with comma-separated env-var lists** (Archisign R2 §3.5) — locked, exactly addresses Archidoc R2 §5.3 ask.

Plus Archidoc's contribution to the consolidated Round-3 doc per the AT9 / Archisign §6 / Archidoc R2 §10 ownership split — see §7.

2. Acceptances — locked in this round

#	Topic	Archidoc's position
AD1	Archisign §3.1 — `envelope.query_resolved` payload schema	N/A for Archidoc directly (handled by Architrak). For Archisign envelopes Archidoc

		itself created, our existing handler logs the event verbatim into `audit_events`.
AD2	Archisign §3.2 — six canonical event names; `query_raised` alias dropped on the wire	Locked. Our verifier whitelist accepts the six canonical names plus `envelope.retention_breach`.
AD3	Archisign §3.3 — seventh event `envelope.retention_breach` + 410 Gone with structured body	Accepted. See §3 below for handling on both paths.
AD4	Archisign §3.4 — rate limits (60 RPM sustained, burst 30, 5,000/day)	Accepted. Archidoc's only outbound to Archisign in this workflow is the re-mint endpoint, called on architect click — far below the budget. Re-mint client will honour `Retry-After` if ever returned.
AD5	Archisign §3.5 — independent key rotation with comma-separated env-var list	Locked. Closes Archidoc R2 §5.3 ask.
AD6	Archisign §4 — three-phase v2 legacy migration adopted verbatim with `ARCHISIGN_WEBHOOK_V2_TENANTS` per-tenant feature flag	Locked. Standard 4–5 week timeline acceptable. We do **not** request the compressed 2.5-week variant Archisign offered — the longer window gives us safer dual-verify telemetry.

AD7	Architrak AT1 — Architrak's verifier whitelist accepts only the six canonical names + the seventh	Confirmed; Archidoc's verifier whitelist is identical.
AD8	Architrak AT2 — `client_checks` columns for the resolver attribution block	N/A for Archidoc.
AD9	Architrak AT5 — `/send` is idempotent on Archisign side	N/A for Archidoc directly; confirms our own re-mint client safety expectations.
AD10	Architrak AT8 — Gmail watcher reads `X-Archidoc-Dqe-Export-Id` case-insensitively at devis ingestion	Confirmed. Our `gmailService.ts` will set the header in lower-case form (`x-archidoc-dqe-export-id`) per RFC 7230 §3.2 — Architrak's case-insensitive read covers any normalisation.
AD11	Architrak AT11 — resend-on-expiry rotates accessToken / re-issues OTP / invalidates old token	N/A for Archidoc directly; relevant only if Archidoc's own e-sign module ever exercises resend (out of scope for the devis workflow).
AD12	Architrak R3 §3 / §8.1 — proactive retention-breach re-notification from Architrak to Archidoc	**Accepted with explicit discriminator and dedup contract.** See §4.
AD13	Architrak R3 §7.3 — insurance-gate semantics	Locked. Mirrors Archidoc R2 §8.1 commitments verbatim.

		Pending only the user's live-gate confirmation.

3. `envelope.retention_breach` — direct-from-Archisign path

For envelopes Archidoc itself created against Archisign (plans, contracts, etc. — pre-existing channel, not the devis workflow), we receive `envelope.retention_breach` directly via our `/api/webhooks/archisign` endpoint after the v2 migration completes. Handling:

- **Verifier whitelist updated** to accept the seventh event name. No other contract change on the existing channel.
- **Side effects on first delivery** (dedup'd by `eventId` against `webhook_events_in`):
  i. Insert one row into a new `signed_pdf_retention_breaches` table (schema in §6 below).
  ii. If the breached envelope corresponds to an existing Archidoc document record (e.g. a stamped contract), append `kind: 'signed_pdf_retention_breach'` to that document's audit trail and surface a red persistent banner on the document detail view.
  iii. If the breached envelope does not correspond to any Archidoc record (orphan), still persist the breach row for forensic completeness — surfaced in a new admin view at Settings → Integrations → "Vault Incidents" (mirrors Architrak's R3 §3 design).
- **No automatic remediation.** Archidoc treats the 410 + `retention_breach` as a definitive vault-side notice. The architect remediates via the `incidentRef` channel out-of-band.
- **Re-mint endpoint behaviour after a breach.** Subsequent calls to `GET /api/v1/envelopes/:envelopeId/signed-pdf-url` for the breached envelope will return 410 with the same `retention_breach` body per Archisign §3.3. Archidoc's re-mint client recognises this status code and surfaces the breach details in the architect's UI rather than retrying.

4. `envelope.retention_breach` — via-Architrak downstream path (direct answer to Architrak R3 §8.1)

Accepted. Proactive notification from Architrak is preferred over lazy discovery on next re-mint attempt, for exactly the reason Architrak gave: the architect should know without having to

click into the document. Specifically:

- **Endpoint**: same as the work-authorisation channel — `POST /api/integrations/architrak/work-authorisations`. We do **not** want a second endpoint; cost and surface area are minimised by reusing one HMAC verifier and one dedup table.
- **Discriminator field**: Architrak adds `eventType: "signed_pdf_retention_breach"` (or `"work_authorised"`, defaulting to the latter when absent for backward-compat with the original payload). Archidoc routes on `eventType` inside the handler.
- **Payload shape for the breach event**:

```
{
 "eventId": "uuid-stable-across-retries",
 "eventType": "signed_pdf_retention_breach",
 "architrakDevisId": 1234,
 "archisignEnvelopeId": "5678",
 "archidocProjectId": "abc-123",
 "incidentRef": "string",
 "remediationContact": "string",
 "originalSignedAt": "ISO-8601",
 "detectedAt": "ISO-8601"
}
```

- **Dedup contract**: same `webhook_events_in` table, dedup on `eventId`. Architrak's stable `eventId` per breach is what we dedup on. A re-delivery returns 200 `{deduplicated: true}` with no side effects.
- **Side effects on first delivery**:
  i. Insert into `signed_pdf_retention_breaches` with `event_source = 'architrak'` and `work_authorisation_id` linked to the existing row.
  ii. Append `kind: 'signed_pdf_retention_breach'` to `projects.auditLog`

for the affected project.

iii. Surface red persistent banner on the project's work-authorisations view, identical pattern to the direct-path handling in §3.

iv. Optional Gmail notification to the project's architect-of-record (gated by a per-tenant setting, default-on).

- **Cross-path dedup**: For the devis workflow specifically, Archidoc receives the breach **only via Architrak's downstream path** — Archisign emits `envelope.retention_breach` only to the `webhookUrl` recorded at envelope create time, which for devis-flow envelopes is Architrak's URL (per Archisign §4.4 and Architrak R2 §A6). So there is **no duplication risk for devis envelopes** in practice. For non-devis envelopes (Archidoc-created plans/contracts), Archidoc receives directly from Archisign per §3 above. The two paths address disjoint envelope sets.
- **Counter-edge case**: if a future architect workflow ever causes Archidoc to create an envelope that ends up in Architrak's devis pipeline (out of scope today), we'd receive the breach on both paths. The `eventId` dedup keeps the second arrival idempotent — but the `event_source` column on the breach row records whichever path arrived first. This is acceptable forensic loss; we can revisit if it ever materialises.

5. Direct answers to remaining counterparty asks

5.1 Archisign R2 §5.1 — acknowledge `envelope.retention_breach` in verifier whitelist

Acknowledged. Already covered by §3 above. Our verifier whitelist (`server/middleware/verifyArchisignWebhook.ts` post-v2 migration) accepts the seven canonical event names exclusively. Anything else returns 401.

5.2 Architrak R3 §8.2 — secrets matrix

Happy for Architrak to produce the first draft per their offer. Archidoc will contribute the rows for `ARCHIDOC_SYNC_API_KEY`, `ARCHIDOC_API_KEY`, `ARCHISIGN_WEBHOOK_SECRET`, `ARCHITRAK_WEBHOOK_SECRET`, `ARCHISIGN_BASE_URL`, plus the new `ARCHISIGN_WEBHOOK_V2_TENANTS` from Archisign §4. Suggested column set per Architrak R3 §8.2 is fine.

6. Revised Archidoc schema (delta from R2 §6)

One addition in this round: the `signed_pdf_retention_breaches` table. Everything else in R2 §6 stands.

```
// shared/schema.ts (additions on top of R2 §6)

export const signedPdfRetentionBreaches =
pgTable("signed_pdf_retention_breaches", {

 id: varchar("id", { length: 255 }).primaryKey(),

 eventId: varchar("event_id", { length: 255 }).notNull().unique(),

 eventSource: varchar("event_source", { length: 32 }).notNull(),  //
'archisign' | 'architrak'

 archisignEnvelopeId: varchar("archisign_envelope_id", { length: 64
}).notNull(),

 workAuthorisationId: varchar("work_authorisation_id", { length: 255
})

   .references(() => workAuthorisations.id, { onDelete: "set null"
}),  // nullable: orphan envelopes are valid

 archidocProjectId: varchar("archidoc_project_id", { length: 255 })

   .references(() => projects.id, { onDelete: "set null" }),

 originalSignedAt: timestamp("original_signed_at").notNull(),

 detectedAt: timestamp("detected_at").notNull(),

 incidentRef: varchar("incident_ref", { length: 255 }).notNull(),

 remediationContact: text("remediation_contact").notNull(),

 receivedAt: timestamp("received_at").defaultNow().notNull(),

 acknowledgedAt: timestamp("acknowledged_at"),

 acknowledgedByUserEmail: varchar("acknowledged_by_user_email", {
length: 255 }),

}, (t) => [

 index("idx_retention_breach_envelope").on(t.archisignEnvelopeId),
```

```
 index("idx_retention_breach_project").on(t.archidocProjectId),

 index("idx_retention_breach_work_auth").on(t.workAuthorisationId),

]);
```

All schema changes (R2 §6 + the table above) ship as one atomic Drizzle migration `drizzle/00XX_devis_signoff_workflow.sql`, gated by Rules 7–10 in our charter.

7. Archidoc's Round-3 contribution to the consolidated doc

Per the AT9 / Archisign §6 / Archidoc R2 §10 ownership split, Archidoc owns the schema-deltas section and the v2 migration timeline. Below is Archidoc's contribution, ready for inclusion in the consolidated Inter-App Contract.

7.1 Cross-app schema-deltas inventory

App	New tables	New columns / enum values	Migration shape
Architrak	`client_checks`, `client_check_messages`, `client_check_tokens`, `insurance_overrides`, `signed_pdf_retention_breaches`, `webhook_deliveries_out`	`signOffStage` enum: `client_review_in_progress`, `client_agreed`, `client_rejected`. `devis`: `archidocDqeExportId`, `archisignEnvelopeId`, `signerIdentityTrail`. `projects`: `clientContactName`, `clientContactEmail`	Single atomic Drizzle migration
Archidoc	`work_authorisations`, `webhook_events_in`, `signed_pdf_retention_breaches`	`insurance_audit_log`: extended JSONB metadata accepts the 6-field `insuranceOverride` block verbatim (no new columns required)	Single atomic Drizzle migration

Archisign	`webhook_deliveries`	`envelopes`: `expiresAt`, `declineReason`, `origin`. New status enum value: `expired`. `signers`: relevant access-token rotation fields (Archisign-internal)	Single atomic Drizzle migration

7.2 Cross-app environment-variable inventory

Variable	Holder app	Other app(s) using	Rotation policy
`ARCHIDOC_SYNC_API_KEY`	Archidoc	Architrak (caller)	Comma-separated list per Archisign §3.5 pattern; Architrak holds copy
`ARCHIDOC_API_KEY`	Archisign	Archidoc (caller)	Comma-separated list per Archisign §3.5; Archisign owns the source of truth
`ARCHITRAK_API_KEY`	Archisign	Architrak (caller)	Comma-separated list per Archisign §3.5; Archisign owns the source of truth

`ARCHITRAK_WEBHOOK_SECRET`	Archidoc	Architrak (signer)	Comma-separated list; rotation coordination via shared secrets vault
`ARCHISIGN_WEBHOOK_SECRET`	Archidoc, Architrak	Archisign (signer)	Same value distributed; rotation coordinated by Archisign + announcement to both receivers
`ARCHISIGN_WEBHOOK_V2_TENANTS`	Archisign	none (Archisign-internal)	Per-tenant feature flag for v2 dual-emit (P1 phase only); cleared at P3 cutover
`ARCHISIGN_BASE_URL`	Archidoc, Architrak	none	Per-environment, no rotation

7.3 v2 signature migration timeline (for the consolidated doc)

Adopting Archisign §4 verbatim with the standard 4–5 week timeline (not the compressed 2.5-week variant):

Phase	Duration	Archisign action	Archidoc action	Architrak action	Risk

P0: Pre-flight	Week 0	Add `ARCHISIGN_WEBHOOK_V2_TENANTS` env var, deploy without enabling	Confirm test environment ready for verifier development	N/A — Architrak is born v2	None
P1: Dual-emit	Weeks 1–2	Enable dual-emit for Archidoc test tenant first, then production tenant after week 1	Verifier ignores unknown headers; baseline logging captures v1 success metrics	N/A	Low
P2: Dual-verify, prefer-v2	Weeks 3–4	Continues dual-emit unchanged	Ship updated `verifyArchisignWebhook.ts`; metric counter `archisign_webhook_verify_path{version}`; require ≥1 week of `v1=0` count	N/A	Medium — verifier bug here breaks Archidoc inbound
P3: Cut-over	Week 5	Drop legacy header; v2-only emission	Verifier hard-requires v2; legacy code path deleted; status code aligned to 401	N/A	Low — by construction P2 has proven v2

					covers 100%

Architrak's new inbound channel ships v2 from day one; no v1/v2 dual-verify on the new `verifyArchitrakWebhook.ts`. Migration cost is absorbed entirely by Archisign (dual-emit) and Archidoc (dual-verify).

7.4 Co-owned shared-helper file

Per Architrak R3 §5 and Archidoc R2 §7 — both apps independently planned to extract a single HMAC helper. Suggestion for the consolidated doc: standardise on the file path `server/middleware/hmacVerifier.ts` (Archidoc's name) or `server/middleware/hmac-verifier.ts` (Architrak's name) per each app's existing naming convention. The math is identical:

```
// pseudo-code for the consolidated spec

function verify({ secret, rawBody, timestampHeader, signatureHeader
}): boolean {

 if (!timestampHeader || !signatureHeader) return false;

 const ts = parseInt(timestampHeader, 10);

 if (Math.abs(Date.now() - ts) > 5 * 60 * 1000) return false;

 const expected = crypto.createHmac('sha256', secret)

   .update(`${timestampHeader}.${rawBody}`)

   .digest('hex');

 const received = signatureHeader.replace(/^sha256=/, '');

 if (expected.length !== received.length) return false;

 return crypto.timingSafeEqual(Buffer.from(expected, 'hex'),
Buffer.from(received, 'hex'));

}
```

Each app implements this signature in their own conventions but the algorithm is byte-identical across all three apps.

8. The one remaining workflow-level blocker

User confirmation of the live-insurance-gate revision. All three agents are aligned:

- **Architrak R2 §6 + R3 §9** — recommends acceptance.
- **Archidoc R2 §3 + this round §2 (AD13)** — non-negotiable from our side.
- **Archisign R2 §S1** — no-op for them, defers to user.

Without this confirmation, the consolidated Round-3 contract cannot be frozen, and three concrete pieces of code cannot be specified: Architrak's `insurance_overrides` table (the two `liveVerdict*` columns), Archidoc's `/api/integrations/architrak/contractors/:contractorId/insurance-verdict` endpoint, and Architrak's outbound retry policy on the live-verdict call.

9. Two flags before you go

- **The new proactive retention-breach re-notification path in §4 introduces one extra side-effect on Archidoc's `/api/integrations/architrak/work-authorisations` endpoint** — same auth, same dedup, but a discriminated payload variant. Worth Architrak knowing this does not break their existing payload shape: the `eventType` field is additive, defaulting to `"work_authorised"` when absent for backward-compat.
- **The user-decision on the live insurance gate is the single workflow-level blocker.** All three agents now formally support overturning the original "mirror-only" lock. If the user holds the line on mirror-only, the contract conflicts and Archidoc will escalate — but everyone has had their say, and the next move is yours.

That's Round 3 from Archidoc. No code or schema changes have been made — this remains a planning artifact only.

Suggested next step for you as the intermediary: compile this Round 3 alongside Architrak's R3 (the doc above already includes Architrak's), then forward both to Archisign for their Round 3. After Archisign closes the loop, the only thing standing between us and a frozen contract is your confirmation on the live-insurance-gate decision.

Once the contract is frozen, I'll come back with a concrete implementation plan for the Archidoc side: the schema migration in §6, the two integration endpoints in R2 §8.1 and §8.2, the v2 Archisign verifier migration in §7.3, the Gmail header emission in R2 §8.5, the re-mint endpoint client in R2 §8.3, and the new `signed_pdf_retention_breaches` table with both ingestion paths from §3 and §4 — laid out as discrete tasks with clean acceptance criteria. You can then choose whether I work them directly or hand them to a task agent.

ARCHISIGN — Round 3 RESPONSE:

ARCHISIGN — Round 3 Response: Devis Sign-Off Workflow Contract

1. Headline summary

The contract is frozen-ready from Archisign's side. With Architrak's R3 (AT1–AT11) and Archidoc's R3 (AD1–AD13) both in hand, every Round 2 position Archisign locked has been accepted by both counterparties — including the new `envelope.retention_breach` seventh event, the v2 HMAC scheme, the rate-limit policy, the independent key rotation semantics, and the three-phase v2 migration timeline.

Three items needed an Archisign position in this round:

1. **Architrak's "no `signOffStage` mutation on retention_breach"** rule (Architrak R3 §3) — affirmed; this is exactly the conservative posture Archisign intended when proposing the event in R2 §3.3. See §3.
2. **Cross-path dedup design for `envelope.retention_breach`** (Architrak R3 §3 + §8.1, Archidoc R3 §4 — the cross-path dedup paragraph) — confirmed as a clean restatement of Archisign's emission rule. Archisign emits the event only once, to the `webhookUrl` recorded at envelope-create time. See §4.
3. **Architrak's peak budget** (R3 §6: 10–25 envelopes per 30-min window, <200/day/tenant) — comfortably inside the published 60-RPM / burst-30 / 5,000-day budget. No adjustment. See §5.

Plus Archisign's contribution to the consolidated Inter-App Contract document per the AT9 / Archidoc §10 / Archisign R2 §6 ownership split (envelope-and-event taxonomy + retention SLA + rate-limit policy + create/send/re-mint orchestration + v2 HMAC scheme). See §7.

The one remaining workflow-level blocker is the user's confirmation on the live-insurance-gate revision. Archisign defers — this is purely between user / Architrak / Archidoc.

2. Acceptances — locked in this round

#	Topic	Archisign's position
AS1	Architrak AT1 / Archidoc AD2 — verifier whitelists locked at six canonical event names + `envelope.retention_breach` (seventh, exceptional)	Locked. Closes Archisign R2 §5.1 ask.
AS2	Architrak AT3 — rate-limit policy honoured with `Retry-After`, queue-depth UI hint	Appreciated. Diagnostic detail in §5 below to make Architrak's UI hint clean.
AS3	Architrak AT11 / R3 §4 — resend-on-expiry semantics (rotated `accessToken`, fresh OTP, invalidated old token)	Confirmed. Closes Archisign R2 §5.2 ask.
AS4	Architrak AT7 / R3 §5 — HMAC scheme `sha256(${timestamp}.${rawBody})` verbatim across all inbound + outbound channels	Locked. Identical to Archisign's v2 implementation.
AS5	Architrak R3 §3 — `envelope.retention_breach` handling (no `signOffStage` mutation, dedicated audit table, red banner, no auto-remediation)	Affirmed. Exactly matches Archisign's intent in R2 §3.3 — see §3 below.

AS6	Architrak R3 §3 / §8.1 + Archidoc AD12 / §4 — proactive re-notification from Architrak to Archidoc with discriminator `eventType: "signed_pdf_retention_breach"`	Acknowledged as Architrak ↔ Archidoc business. Archisign confirms its emission rule does not double-fire — see §4.
AS7	Archidoc AD3 / §3 — direct-from-Archisign retention-breach path on the existing `/api/webhooks/archisign` channel (post-v2-migration) for non-devis envelopes Archidoc itself created	Locked. Same emission rule (one webhook to the create-time `webhookUrl`); plans/contracts envelopes route to Archidoc, devis envelopes route to Architrak. Disjoint envelope sets.
AS8	Archidoc AD6 — standard 4–5 week v2 migration timeline (not the compressed 2.5-week variant Archisign offered)	Locked. The longer dual-emit/dual-verify window is the safer choice; Archisign carries the dual-emit cost without time pressure.
AS9	Architrak AT5 / Archidoc AD9 — `/send` is idempotent against `sent`/`viewed`/`queried`, returns 200 not 409 on safe re-send	Locked. Architrak's outbound retry logic is now safe by contract.
AS10	Architrak AT4 / Archidoc AD5 — independent key rotation with comma-separated env-var lists	Locked. Closes Archidoc R2 §5.3 ask.
AS11	Architrak AT8 / Archidoc AD10 — `X-Archidoc-Dqe-Export-Id` Gmail header, case-insensitive read	N/A for Archisign; noted for the consolidated doc.

AS12	Archidoc R3 §7.4 — pseudo-code for the shared HMAC verifier helper	Byte-identical to Archisign's v2 implementation in `services/WebhookSignature.ts`. Three files (one per app), one algorithm. See §6.
AS13	Architrak R3 §7 + Archidoc R3 §7 — consolidated-doc contributions per the agreed ownership split	Locked. Archisign's section in §7 below completes the three-way split.

3. `envelope.retention_breach` handling — Archisign's posture matches Architrak's §3

Architrak's R3 §3 settled four design points that Archisign R2 §3.3 had implied but not stated explicitly. We now affirm them as part of the frozen contract:

- **No `signOffStage` mutation on the receiver side.** A signed envelope was legally signed; the breach is a future-retrievability problem with the bytes, not a retroactive nullification of the act. Archisign explicitly does not expect either receiver to mark the envelope as anything other than `signed`. This was implicit in R2 §3.3; Architrak made it explicit in R3 §3 first bullet. Locked.
- **`410 Gone` is definitive.** Both `GET /api/v1/envelopes/:envelopeId` and `GET /api/v1/envelopes/:envelopeId/signed-pdf-url` return `410` with the structured `retention_breach` body indefinitely after the breach is detected. No cache, no half-states, no surprise resurrections. Archidoc R3 §3 last bullet correctly anticipates this and instructs their re-mint client to surface the breach details rather than retry.
- **No automatic remediation expected from receivers.** Architrak's "the architect remediates via the `incidentRef` channel out-of-band" (R3 §3) is exactly the receiver behaviour Archisign designed for. We do not expose a remediation API; the `remediationContact` field in the 410 body and in the webhook payload is a human escalation channel.
- **Dedup via stable `eventId`.** A breach event delivered twice (e.g. retry after a transient receiver failure) must be a no-op on the second arrival. Architrak R3 §3 and Archidoc §4 both correctly route the deduplication through the same stable `eventId` table they already maintain. Archisign's outbound retry policy is symmetric: 3 attempts, 1s/3s backoff, 10s per-attempt timeout, identical `eventId` across retries, dead-letter into our

`webhook_deliveries` table after exhaustion.

4. Cross-path dedup — Archisign's emission rule is the answer

Archidoc R3 §4 correctly identifies the dedup logic, and Architrak R3 §3 / §8.1 correctly designs around it. Archisign formalises the rule for the consolidated doc:

Emission rule (locked): Archisign dispatches every envelope-lifecycle webhook — including `envelope.retention_breach` — exactly once per event, to the `webhookUrl` value persisted on the envelope at create time. Archisign does not maintain a registry of "all interested receivers" per envelope, does not consult the caller's tenant identity at emission time, and does not fan out to a list.

Consequences for the devis workflow:

- **Devis envelopes** (created by Architrak, `webhookUrl = ARCHITRAK_BASE/api/webhooks/archisign`) — Archisign emits `envelope.retention_breach` to Architrak only. Architrak's R3 §3 downstream re-notification to Archidoc's `/api/integrations/architrak/work-authorisations` (with `eventType: "signed_pdf_retention_breach"`) is the sole way Archidoc learns about the devis-envelope breach. Single source, single arrival per receiver, no Archisign-side fan-out.
- **Non-devis envelopes** (created by Archidoc, `webhookUrl = ARCHIDOC_BASE/api/webhooks/archisign`) — Archisign emits to Archidoc only. Architrak does not learn about these envelopes from Archisign at all; this is correct, since Architrak has no record of them.
- **Archidoc R3 §4 counter-edge case** (an Archidoc-created envelope ending up in Architrak's devis pipeline) — Archisign's emission rule still routes to the create-time `webhookUrl`. If the workflow ever evolves such that the same logical envelope needs both receivers notified, it remains the responsibility of the receiver who is targeted to re-notify the other; Archisign's contract does not change. The `event_source` column Archidoc plans on `signed_pdf_retention_breaches` is the right forensic detail to capture which path arrived first.

This rule applies symmetrically to all six lifecycle events plus the seventh, not just `retention_breach`. Worth pulling into the consolidated doc explicitly so future contributors do

not re-litigate it.

5. Architrak's peak vs the published budget — direct answer to AT3

Architrak's R3 §6 named: 10–25 envelopes in a 30-minute window from the busiest tenant; <200/day/tenant on the heaviest days. Compare against Archisign R2 §3.4:

- 25 envelopes in 30 minutes ≈ 0.83 RPM sustained, well below the 60-RPM ceiling.
- 25 envelopes in a 5-minute clustered burst (worst realistic Monday-morning scenario) consumes 25 of the 30-burst-token budget — leaves 5 tokens of head-room before any 429, and the bucket refills at 1 token/second.
- 200/day/tenant is 4 % of the 5,000/day ceiling.

No adjustment to the published rate-limit budget. It stands as Archisign R2 §3.4.

Diagnostic commitment to make Architrak's "queue depth" UI hint clean, addressing AT3:

- 429 response body shape: `{ "error": "rate_limit_exceeded", "retryAfter": <seconds>, "limit": "<sustained|burst|daily>", "currentUsage": <int>, "ceiling": <int> }` — so Architrak can show "we hit the burst ceiling, retrying in 12s" rather than an opaque "rate limited."
- `Retry-After` header value matches the body's `retryAfter` field exactly.
- The `X-RateLimit-Remaining` header is set on every 200 response (not just 429s) so Architrak's admin UI can surface a green/amber/red "vault throughput" indicator pre-emptively.

If a tenant ever does breach 5,000/day in production, Archisign will alert (per R2 §3.4's 80% soft alarm) and contact the tenant's owner via `remediationContact`. Architrak's R3 §6 commitment to "coordinate with Archisign before asking for a raise" is the right posture; we'll work it out per-tenant if it ever happens.

6. Shared HMAC verifier — algorithm match

Archidoc R3 §7.4's pseudo-code is byte-identical to Archisign's v2 implementation in `server/services/WebhookSignature.ts`. We confirm:

- Same algorithm shipped on Archisign's v2 webhook **emission** (`createHmac('sha256', secret).update(\`${ts}.${rawBody}\`).digest('hex')`).
- Same algorithm shipped on Archisign's inbound webhook **reception** path (the existing `verifyArchisignWebhook` middleware on Archidoc post-migration; the new

`verifyArchitrakWebhook` middleware on Architrak; and Archisign's own future inbound verifier if any external service ever signs to us).

- File-naming convention divergence is acceptable: Archisign keeps `server/services/WebhookSignature.ts`, Architrak uses `server/middleware/hmac-verifier.ts`, Archidoc uses `server/middleware/hmacVerifier.ts`. Three files, one algorithm. The math is not negotiable; the file path is per-app convention.

The `crypto.timingSafeEqual` call must compare equal-length buffers — Archidoc's pseudo-code correctly guards against length-mismatch before calling it. Lock that detail into the consolidated doc spec.

7. Archisign's contribution to the consolidated Round-3 document

Per AT9 / Archidoc R2 §10 / Archisign R2 §6, Archisign owns the envelope-and-event taxonomy + retention SLA + rate-limit policy + create/send/re-mint orchestration + v2 HMAC scheme.

7.1 Event taxonomy — seven canonical events with full payload shapes

All payloads share a common envelope:

```
{
 "eventId": "uuid-stable-across-retries",          // dedup key
 "event":   "envelope.<name>",                     // canonical name
 "envelopeId": 5678,                               // integer, opaque
to receivers
 "externalRef": "architrak:devis:1234" | null,     // caller-supplied
at create
 "metadata":   { /* verbatim object from create */ } | null,
 "occurredAt": "ISO-8601"
 // ... per-event fields below
}
```

Canonical name	Trigger	Per-event fields
`envelope.sent`	Successful `POST /:envelopeId/send`	`signers: [{ email, name }]`
`envelope.queried`	Signer raises a query in Archisign UI	`queryId`, `signerEmail`, `queryText`, `queriedAt`
`envelope.query_resolved`	Architect resolves the query (Archisign UI or future API)	`queryId`, `resolvedAt`, `resolverSource: "archisign_admin_ui" | "external"`, `resolverEmail | null`, `resolverActor: "architect" | "system"`, `resolutionNote | null`
`envelope.declined`	Signer or admin declines	`declinedBy: signerEmail | "archisign_admin"`, `declinedAt`, `declineReason: string`
`envelope.expired`	Sweeper detects `expiresAt` passed	`expiredAt: ISO-8601` (= the original `expiresAt`)
`envelope.signed`	All required signers signed and PDF stamped	`signedAt`, `signedPdfFetchUrl` (15-min TTL), `signedPdfFetchUrlExpiresAt`, `identityVerification: {…8 fields…}` (per §7.2)

`envelope.retention_breach`	Annual integrity check fails for this envelope	`originalSignedAt`, `detectedAt`, `incidentRef`, `remediationContact`

Lifecycle ordering (informational): `sent` → (`queried` ↔ `query_resolved`)* → terminal (`signed` | `declined` | `expired`). `retention_breach` is post-terminal and only follows `signed`.

7.2 `identityVerification` block — frozen schema

Embedded inside the `envelope.signed` payload:

```
"identityVerification": {

 "method":            "otp_email",                  // enum, single value today

 "otpIssuedAt":       "ISO-8601",

 "otpVerifiedAt":     "ISO-8601",

 "signerIpAddress":   "string",                     // IPv4 or IPv6

 "signerUserAgent":   "string",

 "lastViewedAt":      "ISO-8601",

 "signedAt":          "ISO-8601",

 "authenticationId":  "string"                      // Archisign's own audit-trail FK

}
```

Eight fields, frozen as of R2 §S8. Both receivers persist the block verbatim into a single `jsonb` column without flattening.

7.3 Retention SLA + `retention_breach` error code

- **Retention period:** 10 years from `signedAt` per envelope.
- **Annual integrity check:** Archisign runs an integrity job once per calendar year that walks the vault and verifies every retained envelope is retrievable. Envelopes that fail the check are marked `retention_breach` internally.

- **Read-side error code on breached envelopes:** `GET /api/v1/envelopes/:envelopeId` and `GET /api/v1/envelopes/:envelopeId/signed-pdf-url` return:

```
HTTP/1.1 410 Gone
Content-Type: application/json
{
 "error":              "retention_breach",
 "envelopeId":         5678,
 "originalSignedAt":   "ISO-8601",
 "detectedAt":         "ISO-8601",
 "incidentRef":        "string",
 "remediationContact": "string"
}
```

- **One-shot webhook on breach detection:** Archisign emits `envelope.retention_breach` (per §7.1) to the `webhookUrl` recorded at envelope-create time. Per §4 above, this is a single emission to a single receiver; cross-receiver propagation is the receivers' problem.
- **`410` is not `404`:** the legal distinction matters. `404` says "we never had this." `410` + `retention_breach` says "we had it, we know it was signed at `originalSignedAt`, and we know the bytes are no longer retrievable as of `detectedAt`." This preserves the audit position even when the artefact is gone.

7.4 Rate-limit policy

Per §5 above (no change from R2 §3.4):

Bucket	Limit	Per
Sustained	60 requests / minute	(API key, endpoint family)
Burst	30 tokens, refilling at 1 token / second	(API key, endpoint family)

Daily	5,000 requests / day	(API key, endpoint family)

Endpoint families: `create` (POST /api/v1/envelopes/create), `send` (POST /:envelopeId/send), `read` (GET /:envelopeId, GET /:envelopeId/signed-pdf-url). Each family has independent counters per key.

429 response shape and `Retry-After` / `X-RateLimit-Remaining` headers per §5.

7.5 Two-step orchestration — endpoint contracts

```
POST /api/v1/envelopes/create

Header: X-API-KEY: <ARCHITRAK_API_KEY | ARCHIDOC_API_KEY>

Body:   { pdfFetchUrl, externalRef?, metadata?, signers[], fields[],

         webhookUrl, expiresAt?, identityVerification: { method } }

Response 201: { envelopeId, status: "draft", createdAt, expiresAt,

               signers: [{ id, accessToken, accessUrl, otpDestination
}] }

401 invalid X-API-KEY · 400 pdfFetchUrl unfetchable within 60s budget
· 413 PDF > 25 MiB

429 rate-limited · 503 vault transient

POST /api/v1/envelopes/:envelopeId/send

Header: X-API-KEY: <same key as create>

Body:   {} (idempotent on the send side)

Response 200 on first send AND on re-send against status ∈ {sent,
viewed, queried}:

                { envelopeId, status, sentAt }

401 · 404 envelope not found · 409 envelope status ∈ {signed,
```

```
declined, expired, void}

429 · 503

GET /api/v1/envelopes/:envelopeId/signed-pdf-url

Header: X-API-KEY: <ARCHITRAK_API_KEY | ARCHIDOC_API_KEY>

Response 200: { url, expiresAt }   // 15-min TTL signed URL

401 · 404 · 410 retention_breach (per §7.3) · 429 · 503
```

Both API keys can call all three endpoints for any envelope they can identify; no per-key envelope ownership enforcement. The `metadata` blob round-trips verbatim on every webhook. The `externalRef` is opaque to Archisign and is the receiver's join key against their own database.

7.6 v2 HMAC scheme — wire spec

- **Algorithm:** HMAC-SHA256, hex-encoded.
- **Signed material:** `${X-Archisign-Timestamp}.${rawBody}` (literal `.` joiner; raw request bytes, not re-stringified parsed JSON; timestamp is decimal unix-ms string).
- **Headers on every Archisign-emitted webhook:**
  - `X-Archisign-Signature: sha256=<hex>` (post-v2; legacy `X-Archisign-Signature` co-emitted during the migration P1 phase per §7.7)
  - `X-Archisign-Timestamp: <unix ms>`
- **Replay window:** ±5 minutes server time, enforced via `crypto.timingSafeEqual` on the recomputed signature.
- **Receiver error codes:** missing/malformed header → 401 (non-retryable); stale timestamp → 401; signature mismatch → 401 (Archidoc's existing 403 aligns to 401 at P3 cutover per their R2 §4).
- **Body size cap:** 1 MiB → 413.
- **Header on every Architrak/Archidoc-emitted webhook to Archisign** (symmetric, but Archisign is not a receiver in this workflow): same scheme would apply.

7.7 v2 migration timeline — adopting Archidoc R3 §7.3 verbatim

Standard 4–5 week timeline (not the compressed 2.5-week variant). Archisign carries the dual-emit cost on the legacy `/api/webhooks/archisign` channel to Archidoc; Architrak's new channel is born v2-only. Per-tenant feature flag `ARCHISIGN_WEBHOOK_V2_TENANTS` controls

dual-emit during P1.

(See Archidoc R3 §7.3 for the full week-by-week table — Archisign confirms each row.)

8. Archisign's contribution to the secrets matrix

Architrak R3 §8.2 offered to draft; Archidoc R3 §5.2 accepted. Archisign's rows for the Architrak-led draft:

Variable	Holder app (source of truth)	Other app(s) using	Rotation policy
`ARCHIDOC_API_KEY`	Archisign	Archidoc (caller)	Comma-separated list per Archisign R2 §3.5; rotation initiated by Archisign, copy distributed to Archidoc out-of-band; revocation does not affect `ARCHITRAK_API_KEY`
`ARCHITRAK_API_KEY`	Archisign	Architrak (caller)	Comma-separated list per Archisign R2 §3.5; same rotation pattern; independent of `ARCHIDOC_API_KEY`

`ARCHISIGN_WEBHOOK_SECRET`	Archisign (single value, distributed)	Archidoc, Architrak (signers/receivers)	Same value distributed to both receivers; rotation announced by Archisign with a 2-week dual-secret window where both old and new are accepted; comma-separated list on receiver side during the window
`ARCHISIGN_WEBHOOK_V2_TENANTS`	Archisign	none (Archisign-internal)	Per-tenant feature flag for v2 dual-emit during P1; cleared at P3 cutover; not a secret per se — flag value, no rotation
`ARCHISIGN_BASE_URL`	Archisign (publishes)	Archidoc, Architrak (callers)	Per-environment, no rotation

9. Outstanding asks from Archisign for the consolidated doc

None from Archisign. Every R2 ask has been answered in this round. The same single workflow-level blocker remains:

User confirmation of the live-insurance-gate revision. All three agents are aligned (Architrak R3 §9, Archidoc R3 §8, Archisign R2 §S1). Archisign defers — this is purely between user / Architrak / Archidoc. Without confirmation, the consolidated Round-3 contract cannot be frozen.

10. Two flags before close

- **The cross-path dedup rule in §4** is, strictly speaking, a restatement of an

Archisign-side emission rule that has been implicit since R1. Pulling it explicitly into the consolidated doc — under Archisign's section — closes a question that Architrak and Archidoc were independently designing around in R3 §3/§8.1 and §4 respectively. Without it, future contributors might assume Archisign fans out to all interested receivers per envelope; that is not the contract.

- **The contract is otherwise frozen-ready.** The consolidated Inter-App Contract document can be assembled within hours of the user's live-insurance-gate confirmation, with three sections (Architrak's lifecycle in their R3 §7; Archidoc's schema-deltas + v2 timeline in their R3 §7; Archisign's envelope-and-event taxonomy + SLA + rate limits + orchestration + HMAC scheme above in §7) plus the three-way co-owned secrets matrix using Archisign's rows in §8 as input.

That's Round 3 from Archisign. No code or schema changes have been made — this remains a planning artifact only.

Suggested next step for you as the intermediary: (a) confirm the live-insurance-gate decision, then (b) assemble the consolidated Inter-App Contract document from the three R3 contributions (Architrak §7, Archidoc §7, Archisign §7 above) plus the secrets matrix.

Once the contract is frozen, I'll come back with a concrete implementation plan for the Archisign side: the `ARCHITRAK_API_KEY` middleware extension, the `pdfFetchUrl` server-side fetch service, the four new wire events (`envelope.declined`, `envelope.expired`, `envelope.query_resolved`, `envelope.retention_breach`) plus the `envelope.queried`-payload upgrade, the `webhook_deliveries` idempotency table + dead-letter UI, the v2 signature scheme + dual-emit migration with the `ARCHISIGN_WEBHOOK_V2_TENANTS` flag, the `expiresAt` sweeper, the re-mint endpoint, the rate-limit middleware with the diagnostic `X-RateLimit-Remaining` header, and the annual integrity check job — laid out as discrete tasks with clean acceptance criteria. You'll then choose whether I work them directly or hand them to a task agent.

Worked for 4 minutes

Scroll to latest